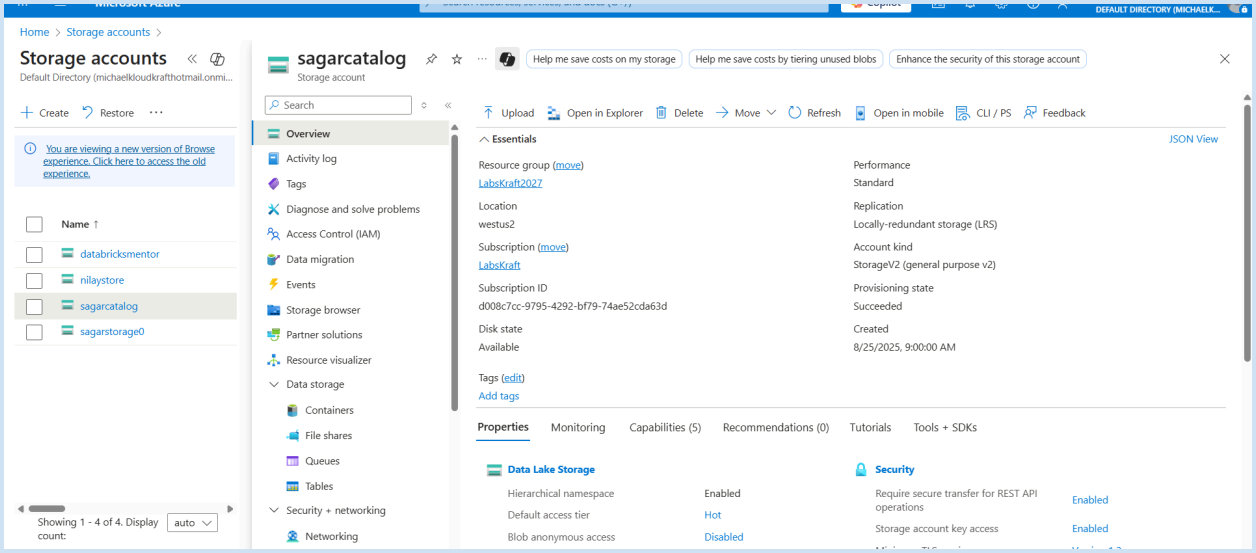
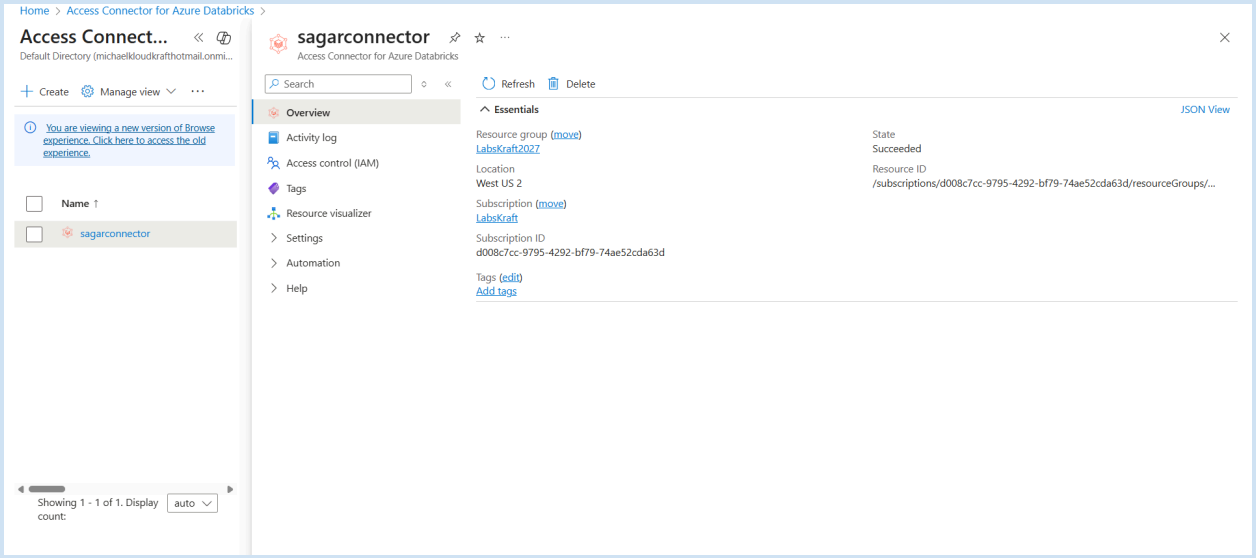
IoT-Enabled Retail Analytics on Azure Databricks

Resources Creation

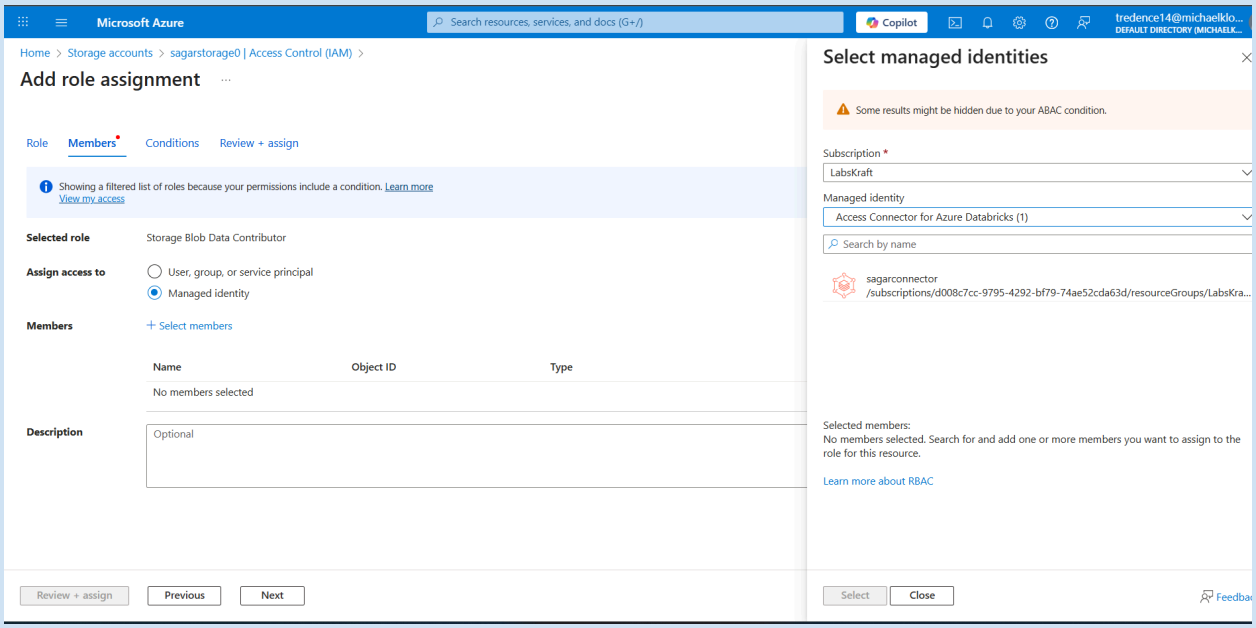
creation of storage account:



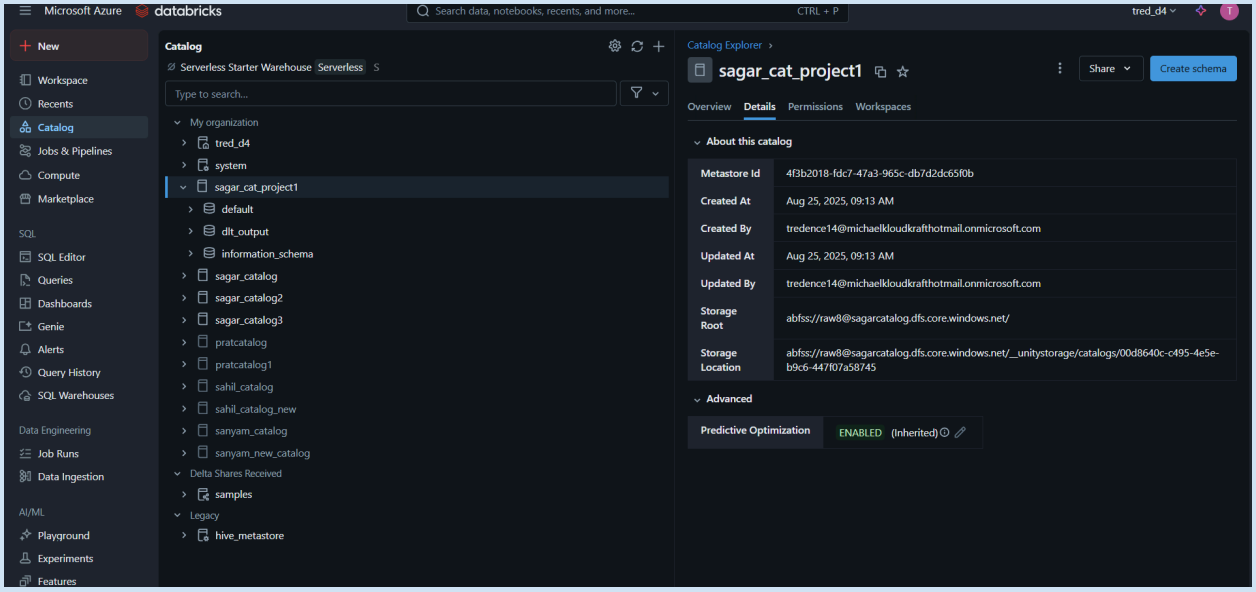
creation of access connector:



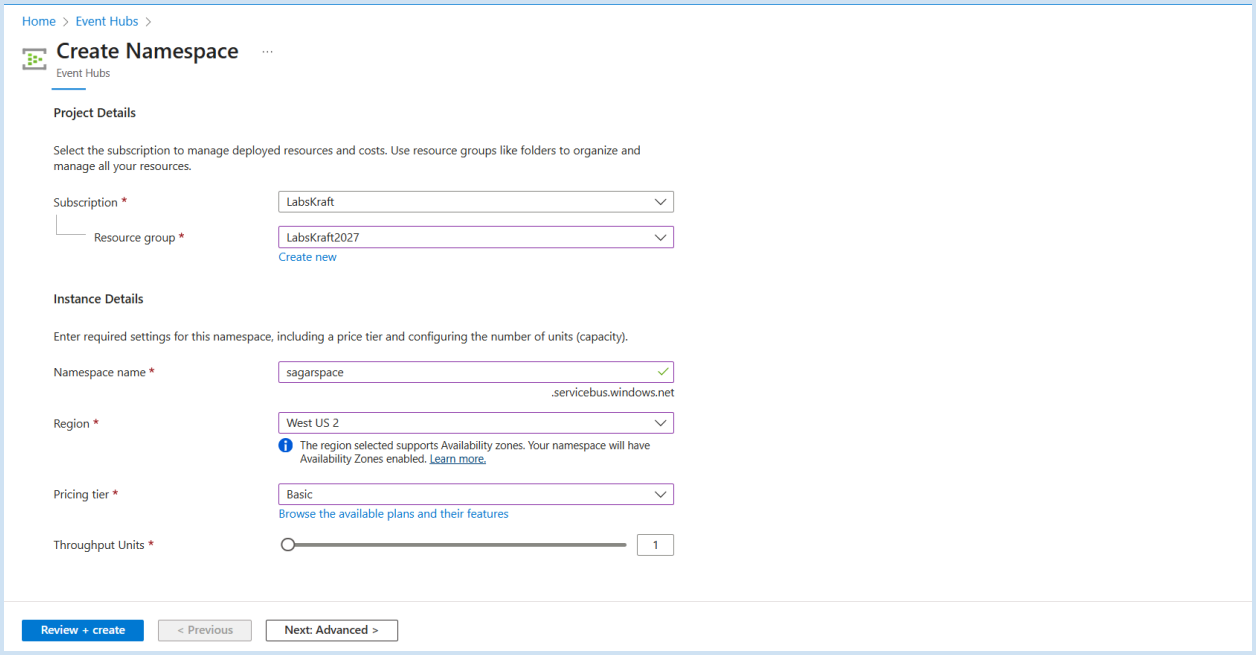
giving access connector permission for storage:

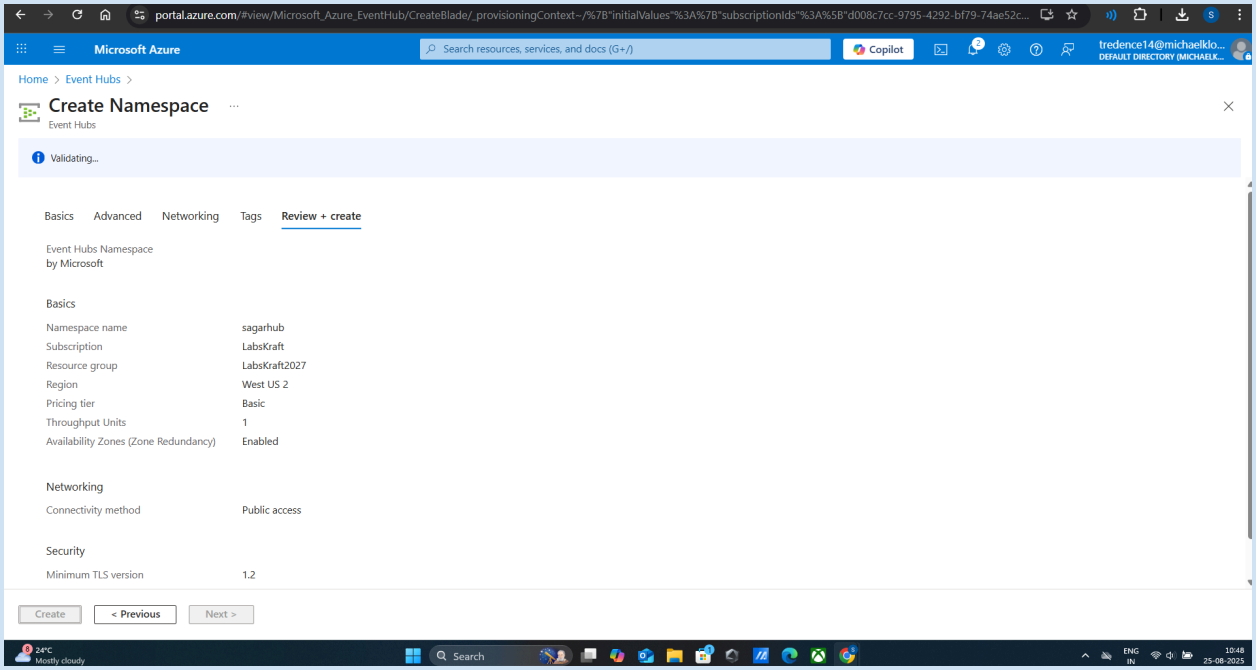


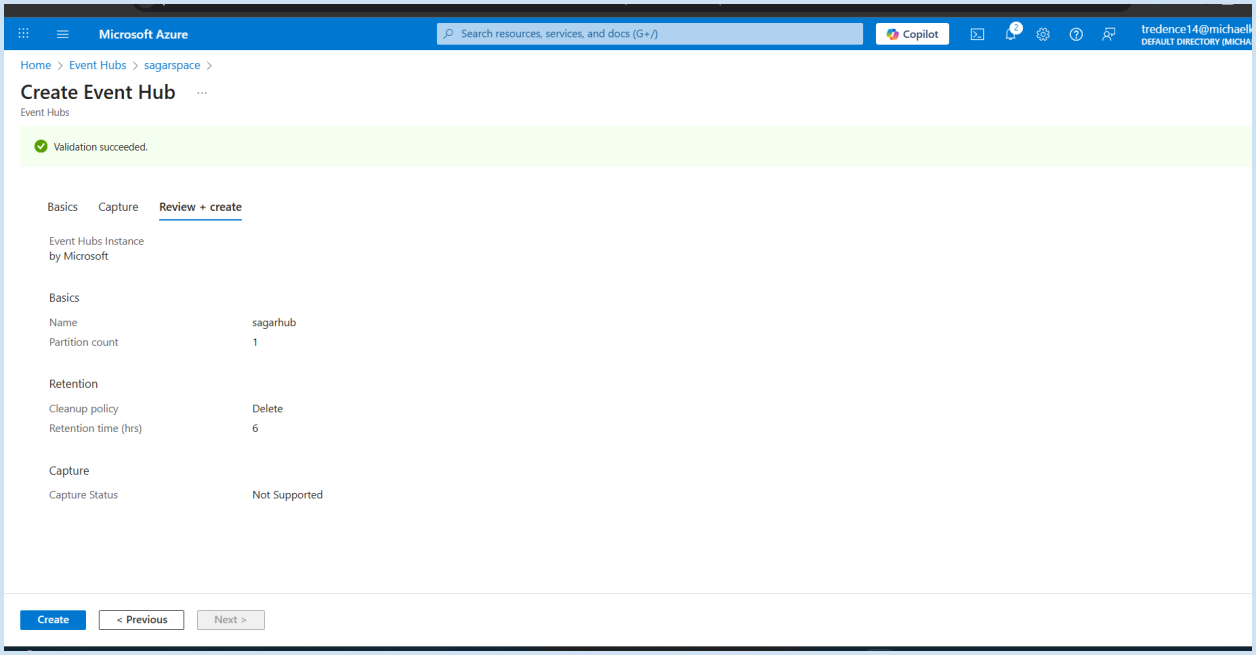
creation of uc:



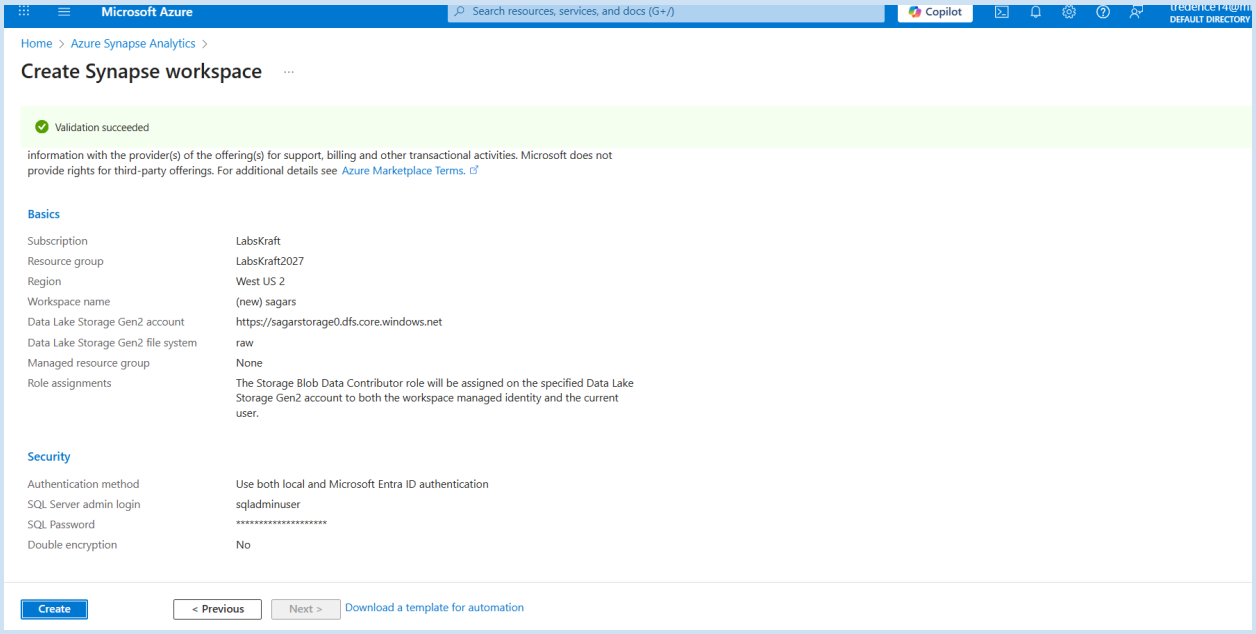
creation of namespace and eventhub:




portal.azure.com/#view/Microsoft_Azure_EventHub/CreateBlade/_provisioningContext~/%7B"initialValues"%3A%7B"subscriptionIds"%3A%5B"d008c7cc-9795-4292-bf79-74ae52c...
Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
tredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELK...
Home > Event Hubs >
Create Namespace
Event Hubs
Validating...
Basics Advanced Networking Tags Review + create
Event Hubs Namespace
by Microsoft
Basics
Namespace name sagarhub
Subscription LabsKraft
Resource group LabsKraft2027
Region West US 2
Pricing tier Basic
Throughput Units 1
Availability Zones (Zone Redundancy) Enabled
Networking
Connectivity method Public access
Security
Minimum TLS version 1.2
Create
< Previous
Next >
24°C
Mostly cloudy
Search
ENG
IN
10:48
25-08-2025

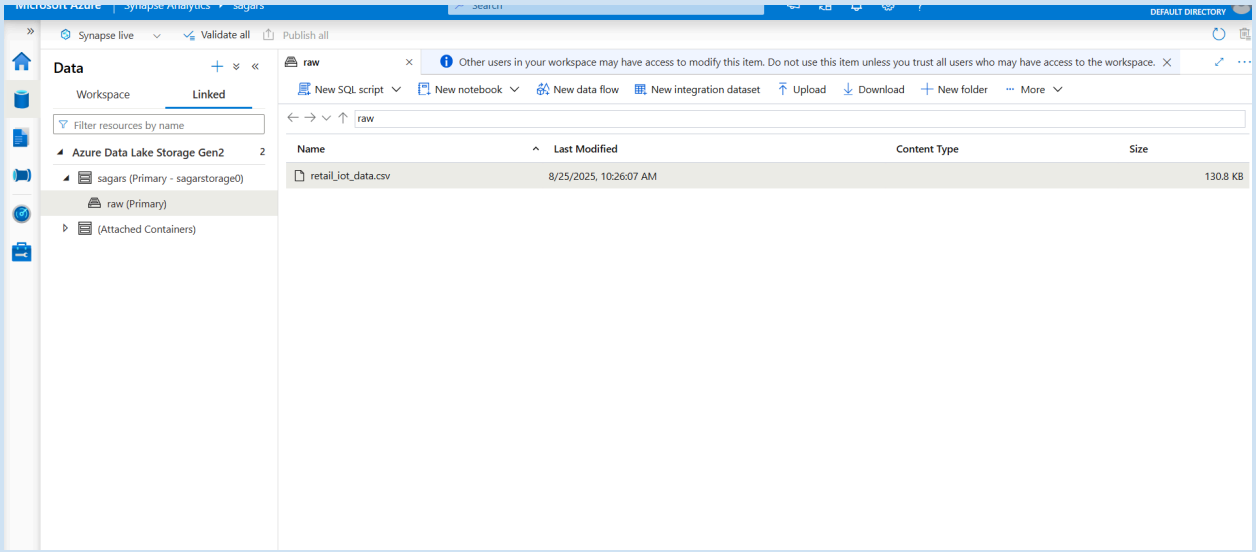

Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
tredence14@michaelk
DEFAULT DIRECTORY (MICHA
Home > Event Hubs > sagarspace >
Create Event Hub
Event Hubs
Validation succeeded.
Basics Capture Review + create
Event Hubs Instance
by Microsoft
Basics
Name sagarhub
Partition count 1
Retention
Cleanup policy Delete
Retention time (hrs) 6
Capture
Capture Status Not Supported
Create
< Previous
Next >

creating synapse workspace:

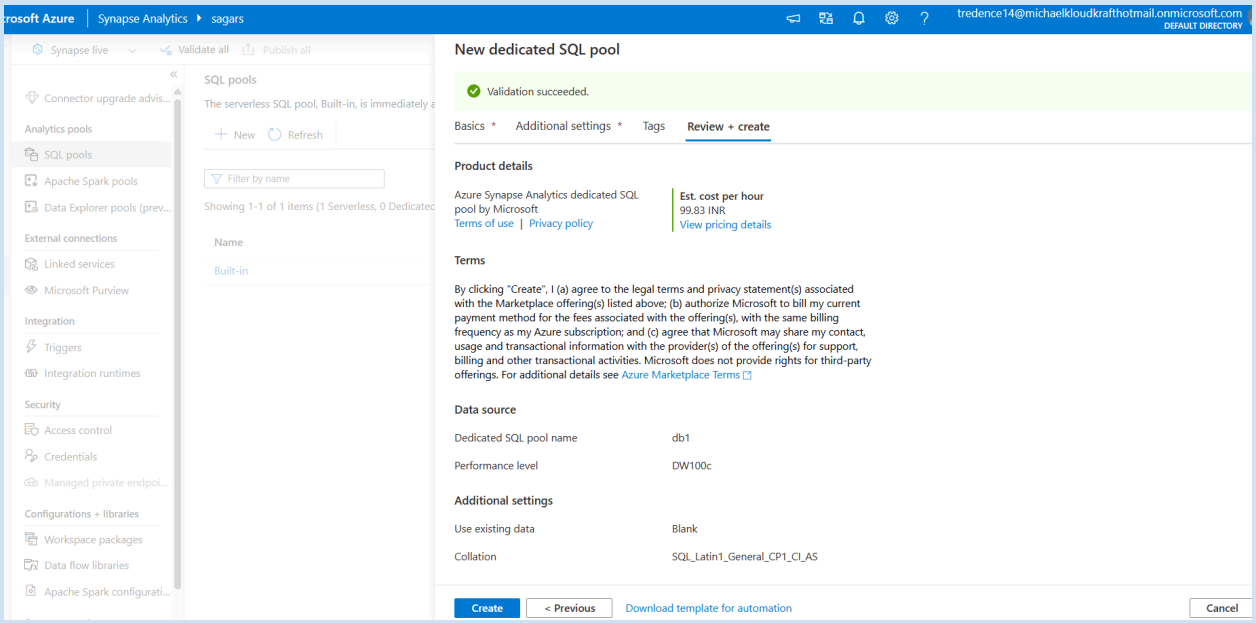


setting up synapse ->

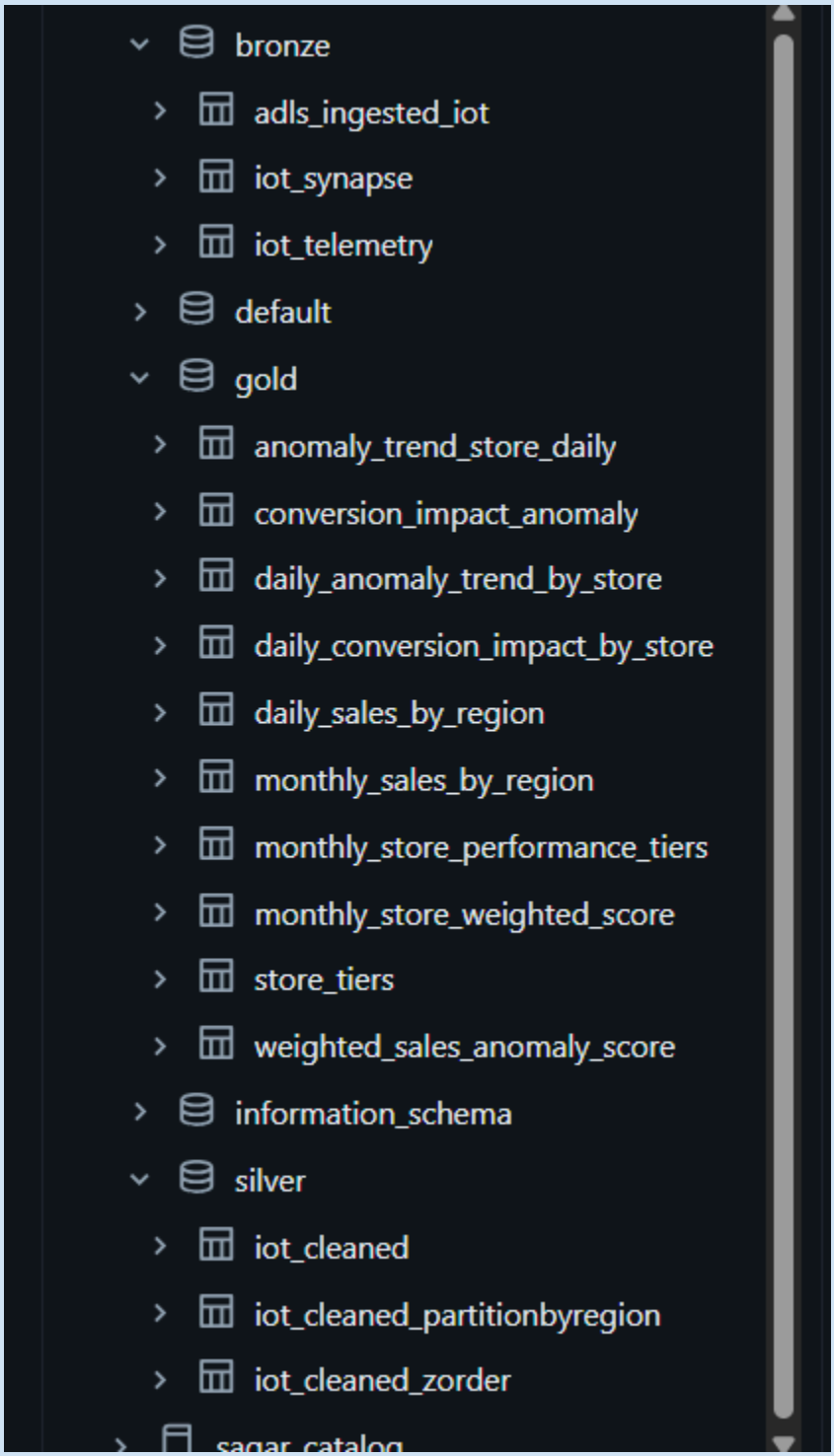
linking my adls for storing old data and these can be pulled into azure dbx



creation of new dedicated pool:

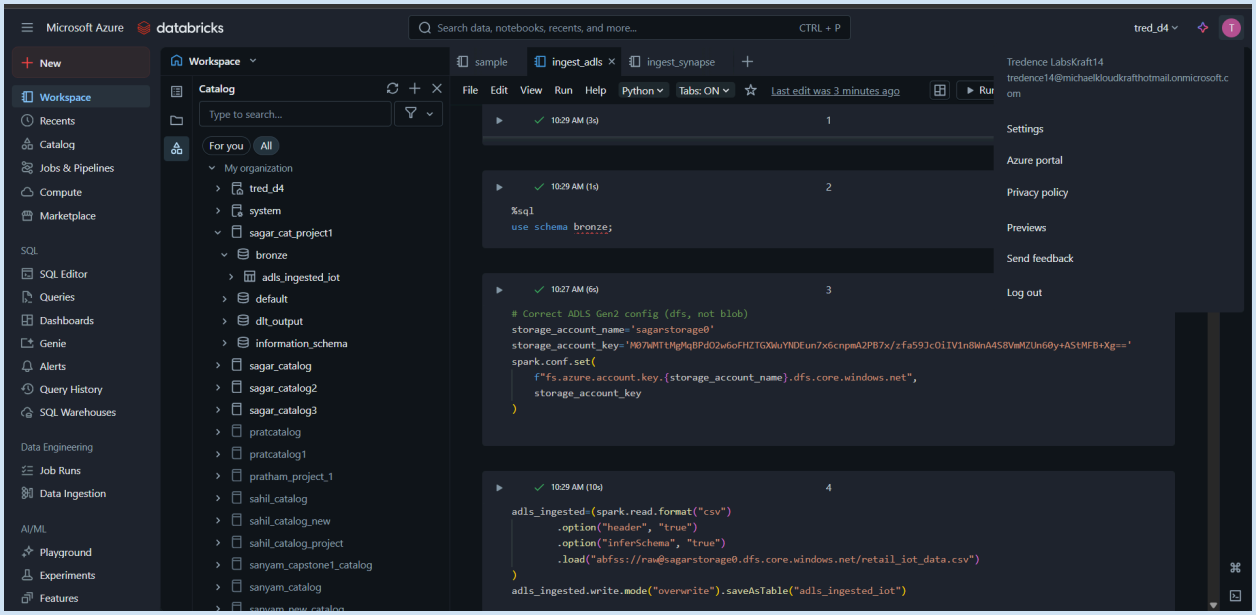


Quick Sumary->

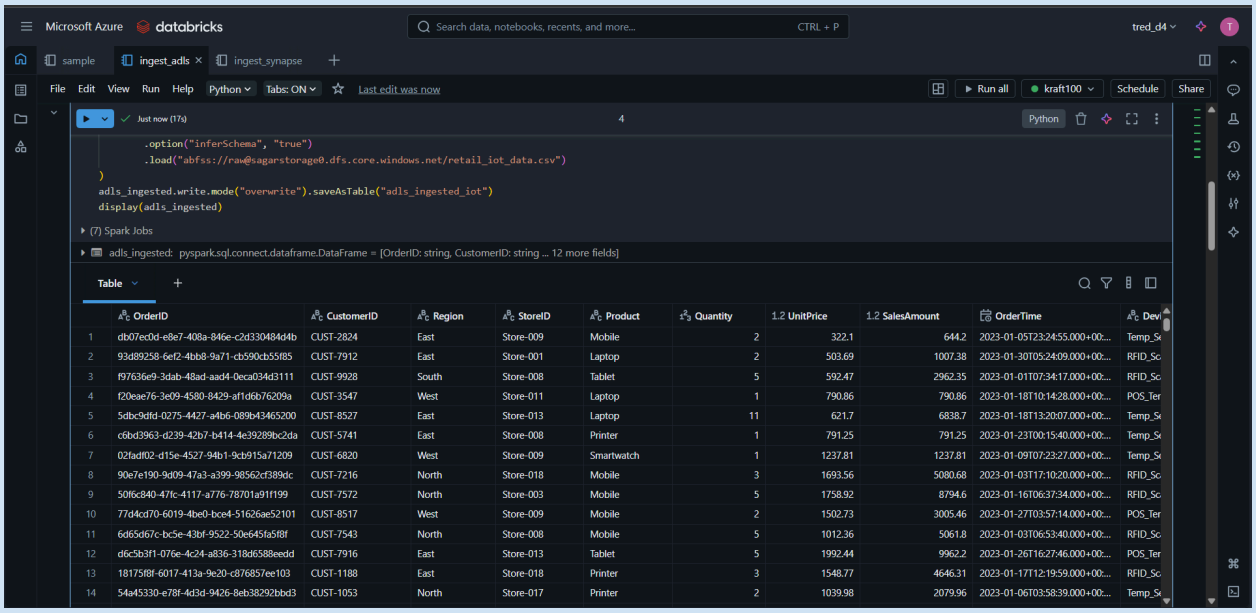


Ingestion mode

i) ADLS
adls connection and storing file in sagar_cat_project1.bronze schema

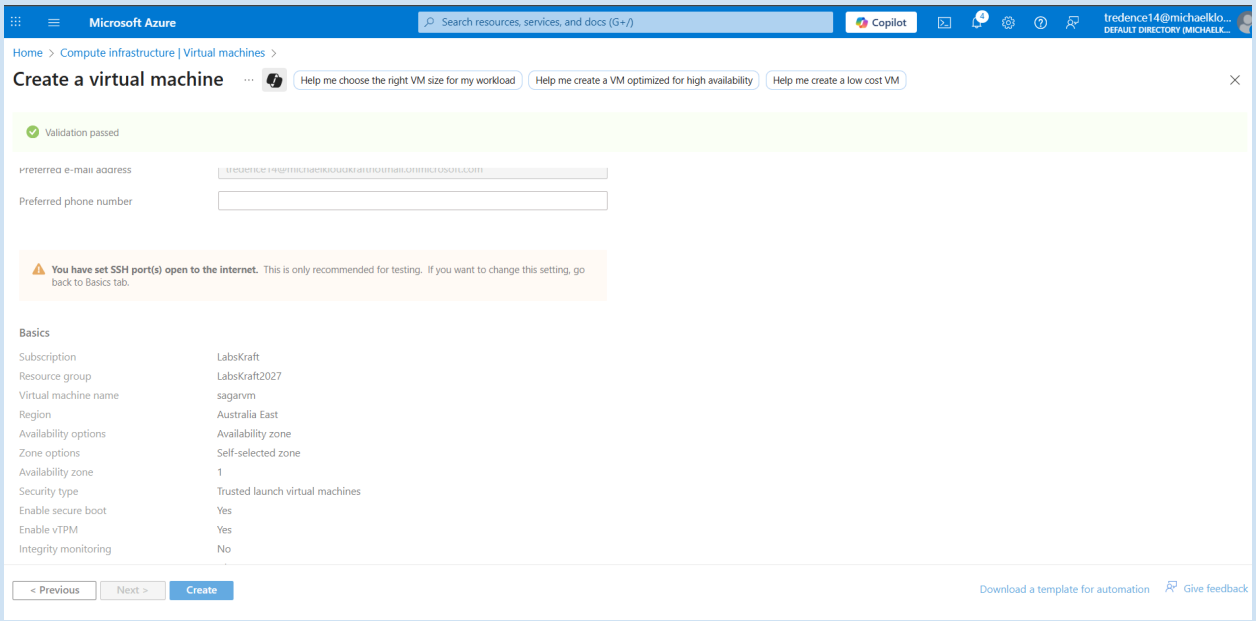


adls_ingested_table->

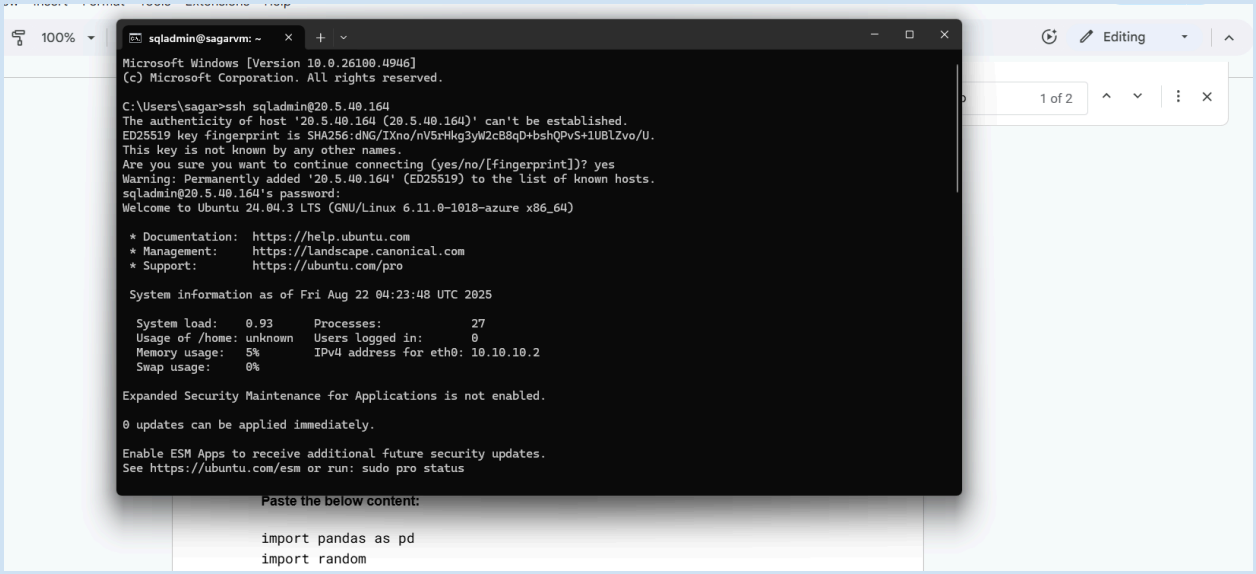


ii)Event Hub:
we will connect to vm and then send data from local machine to eventhub which at the end will be recieved by DBX

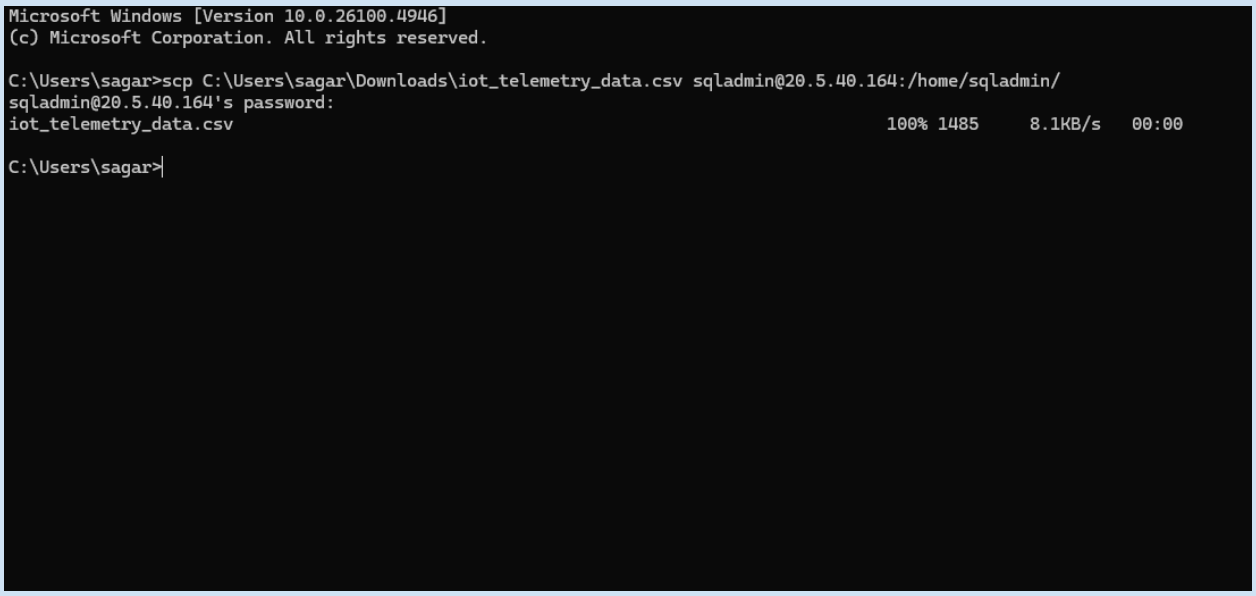
creation of vm (namespace and eventhub already created) ->



setting up vm session on local machine->



uploading my file to vm which is considered for ingestion



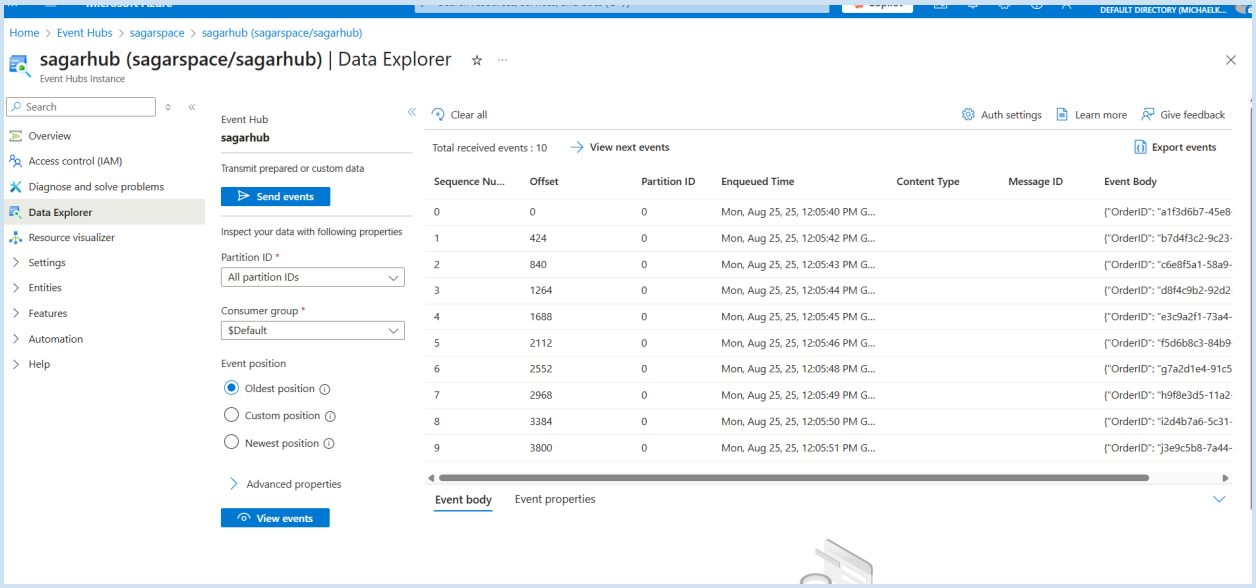
nano send.py

```
  GNU nano 7.2                                   send.py
ls /home/sqladmin/from azure.eventhub import EventHubProducerClient, EventData
import pandas as pd
import time
import json

# Load the CSV
df = pd.read_csv("iot_telemetry_data.csv")

# Event Hub connection
connection_str = "Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=2v07F8+tBbuMO2W5qjzsSpUAVMNbYgfeZ+AEhBrynDE="
eventhub_name = "sagarhub"

producer = EventHubProducerClient.from_connection_string(conn_str=connection_str, eventhub_name=eventhub_name)

# Stream data row by row
for _, row in df.iterrows():
    data = row.to_dict()
    event_data_batch = producer.create_batch()
    event_data_batch.add(EventData(json.dumps(data)))
    producer.send_batch(event_data_batch)
    print(f"Sent: {data['OrderID']}")
    time.sleep(1)  # simulate 1 event/sec
```

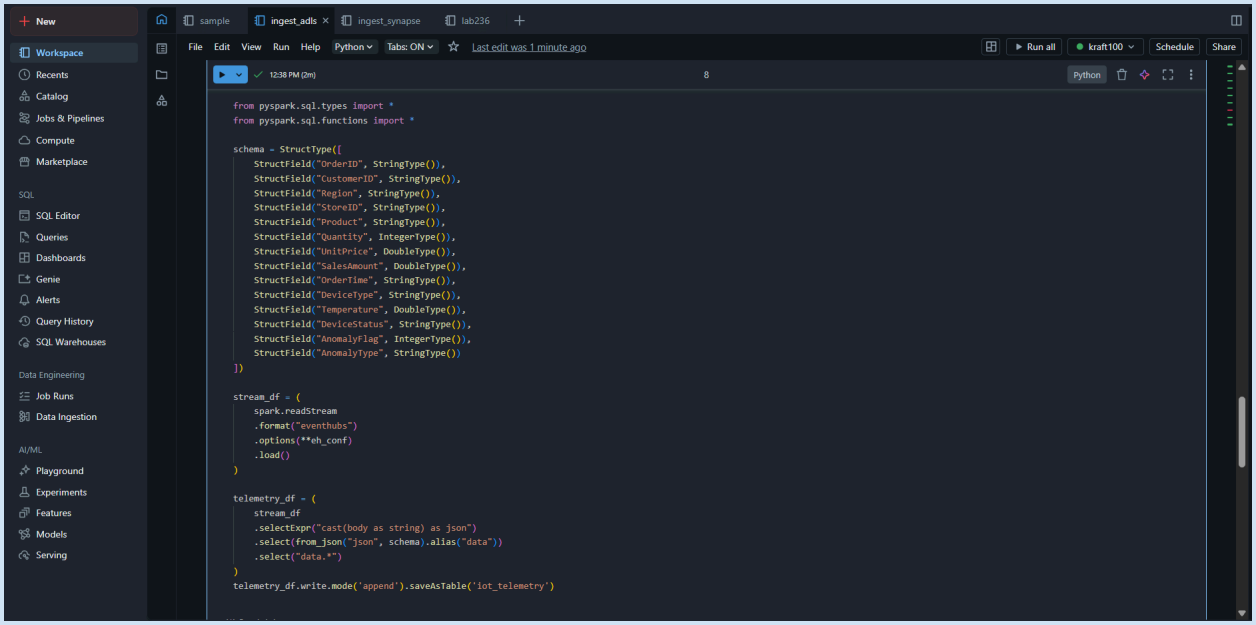
sent file:

```
  WARNING: The script faker is installed in '/home/sqladmin/.local/bin' which is not
  Consider adding this directory to PATH or, if you prefer to suppress this warning,
Successfully installed faker-37.5.3 numpy-2.3.2 pandas-2.3.2 tzdata-2025.2
sqladmin@sagarvm:~$ python3 send.py
Sent: a1f3d6b7-45e8-4a30-8e9b-1f9d2caa6710
Sent: b7d4f3c2-9c23-4a0a-a4b5-f0317b53f8f5
Sent: c6e8f5a1-58a9-4980-a71c-34266c5db412
Sent: d8f4c9b2-92d2-4a9f-a6b0-15db9d214e62
Sent: e3c9a2f1-73a4-41e8-b4fd-62b0a930ed7b
Sent: f5d6b8c3-84b9-4a77-a9c1-9d1e1cbf6d2e
Sent: g7a2d1e4-91c5-4f21-bf28-7bc0e55d9183
Sent: h9f8e3d5-11a2-48a4-9c2b-64b0b11c6d45
Sent: i2d4b7a6-5c31-4c29-bd38-27f7a04d0c9e
```

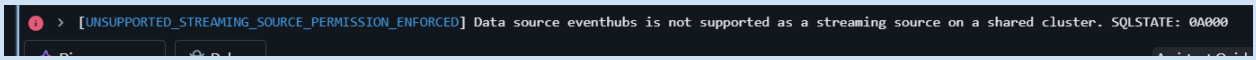
File received on EH



Reading data from eventhub and storing into my bronze schema as iot_telemetry

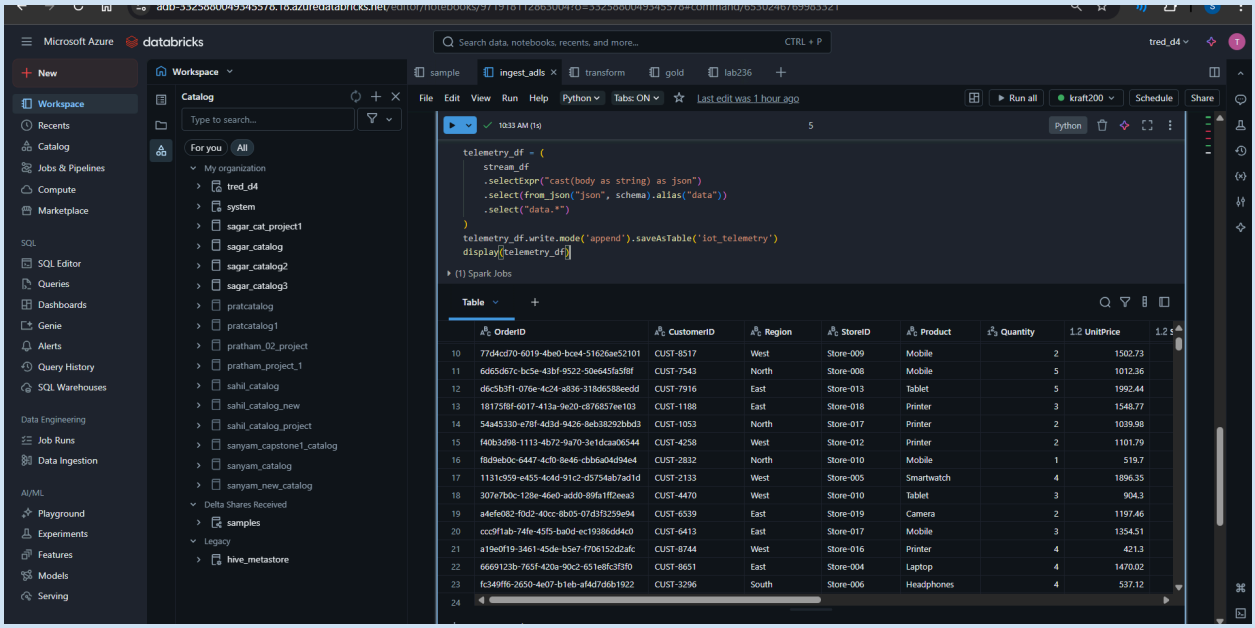


error i got->



i solved by making personal cluster .

Data streamed in from event_hub->



Both table from adls and eventhub stored in my bronze schema of catalog.
iii) Azure synapse ingestion ->

```python
# Synapse connection details
synapse_workspace = "sagars"
synapse_pool = "db1" # Your dedicated SQL pool name
synapse_user = 'sqladmin'
synapse_password = ''
synapse_jdbc_url = f"jdbc:sqlserver://{synapse_workspace}.sql.azuresynapse.net:1433;database={synapse_pool};"

# Temporary staging location on ADLS Gen2
# This MUST be a storage account accessible by both Synapse and Databricks
temp_storage_account = "sagarstorage0"
temp_container = "syn"
temp_dir = f"abfss://{temp_container}@{temp_storage_account}.dfs.core.windows.net/synapse-dbx-staging"

# Source and Target table names
source_table = "dbo.RetailSales"
target_catalog = "sagar_cat_project1"
target_schema = "bronze"
target_table_name = f"{target_catalog}.{target_schema}.iot_syn"


# 2. Read from Synapse using the connector
print(f"Reading data from Synapse table: {source_table}...")
df = (spark.read
  .format("com.microsoft.sqlserver.jdbc.spark") # Use the Synapse connector
  .option("url", synapse_jdbc_url)
  .option("dbtable", source_table)
  .option("user", synapse_user)
  .option("password", synapse_password)
  .option("tempDir", temp_dir) # Crucial for performance
  .option("forwardSparkAzureStorageCredentials", "true")
  .load()
)

# 3. Write the data as a Delta table in Databricks
print(f"Writing data to Delta table: {target_table_name}...")
(df.write
   .format("delta")
   .mode("overwrite")
   .saveAsTable(target_table_name)
)

print("Import complete!")
```

```python
display(spark.table('sagar_cat_project1.bronze.iot_syn'))
```

▸ (4) Spark Jobs

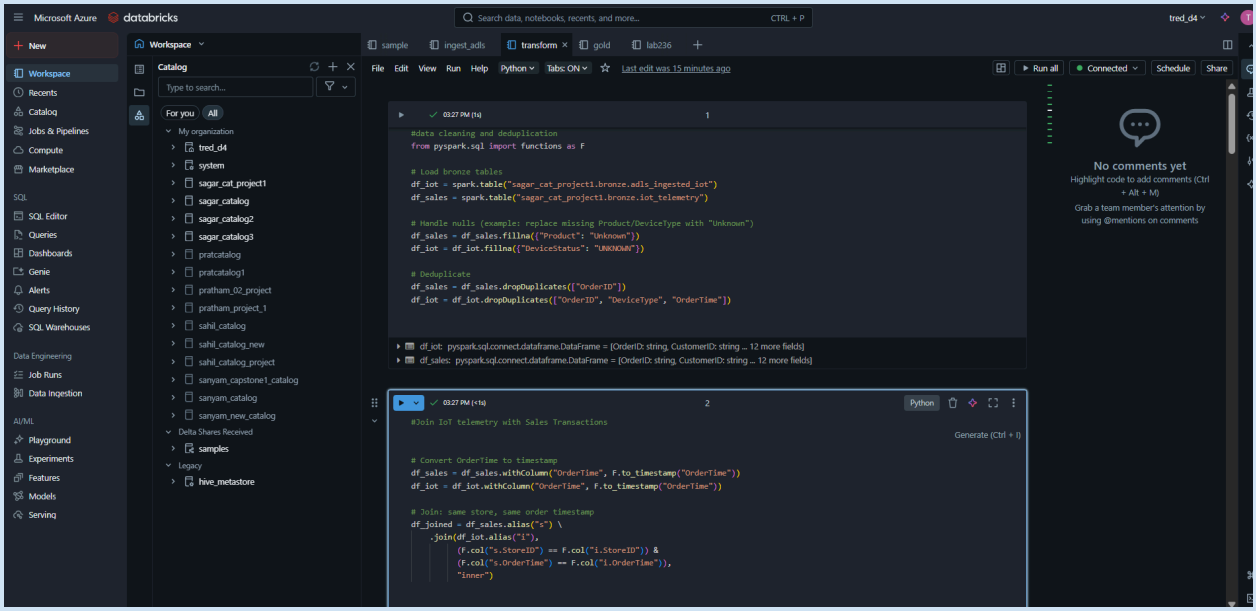
▸ adls_ingested: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

Table +

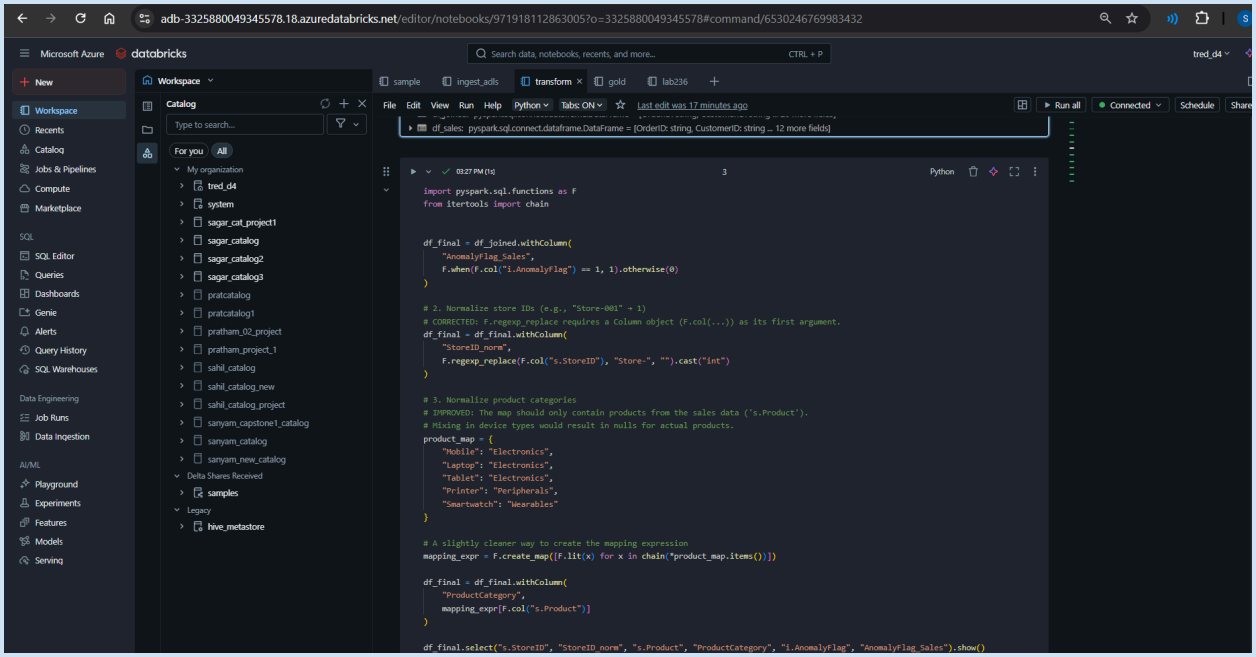
	OrderID	CustomerID	Region	StoreID	Product	Quantity
57	9a72be7d-3ca5-4b64-878c-f443aea6cae5	CUST-4613	South	Store-008	Headphones	4
58	9fad8b36-fcd2-4037-a019-728e16ca3686	CUST-3704	South	Store-005	Camera	13
59	199de6e5-8d11-4887-b2b7-c05dab1b01...	CUST-3223	South	Store-006	Laptop	3
60	8b67fa52-42bb-486a-866c-aa1a893ee151	CUST-7211	North	Store-014	Headphones	1
61	3afae91f-cc69-46ec-941b-58cefcb85d7e	CUST-2124	East	Store-001	Mobile	2
62	fae848cf-90c7-46fe-af09-87e58efbf673	CUST-4571	South	Store-009	Printer	3
63	14dc706f-5b8c-45ff-98a3-5246959b1b69	CUST-3683	North	Store-004	Camera	1
64	2b4da48a-9939-4bb3-8203-0f35369dff71	CUST-4249	East	Store-019	Smartwatch	2
65	2bb8c321-2f07-408d-895a-217cda90ee50	CUST-1672	North	Store-018	Tablet	3
66	82961f57-063d-452e-abe1-c8c0ce076f1d	CUST-2729	South	Store-012	Smartwatch	4
67	cd669277-e660-44cd-82ad-a365c6d375cd	CUST-5425	South	Store-015	Tablet	5
68	22c7aca4-9dda-4759-b9b9-2c585ae3cc71	CUST-8613	South	Store-011	Laptop	4
69	37ce4b04-72d4-4b8e-ad0f-0d98e56445a0	CUST-6576	North	Store-020	Smartwatch	3
70	5e8bc1da-b338-4da5-a533-e79d55161bc9	CUST-9004	West	Store-016	Smartwatch	4
71						

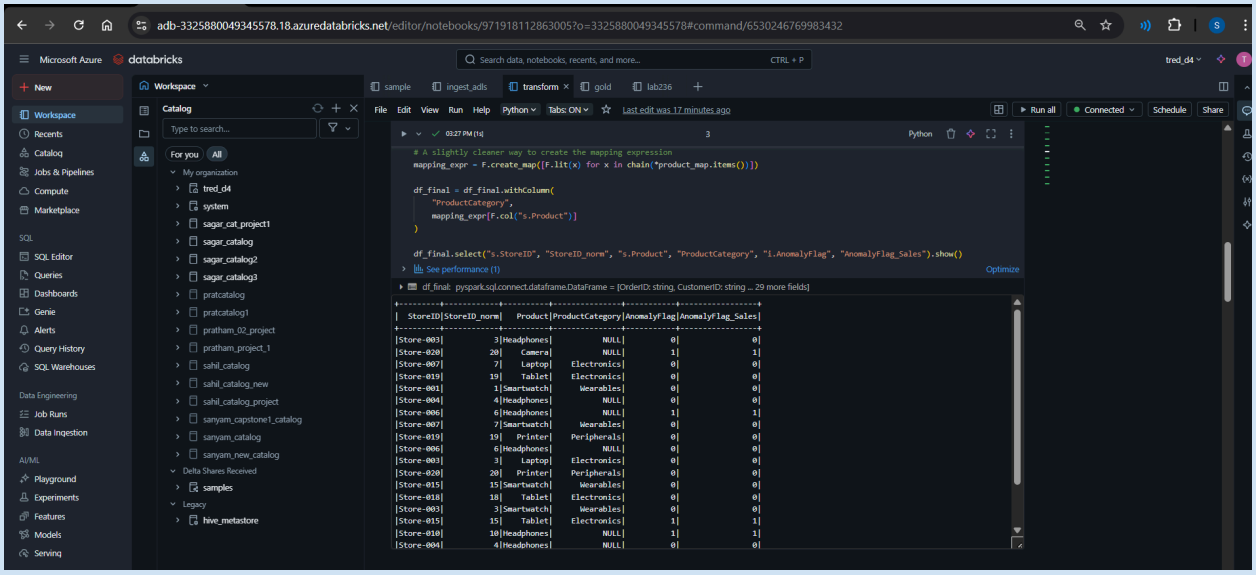
Transformation:

Removal of data quality issues and renaming column



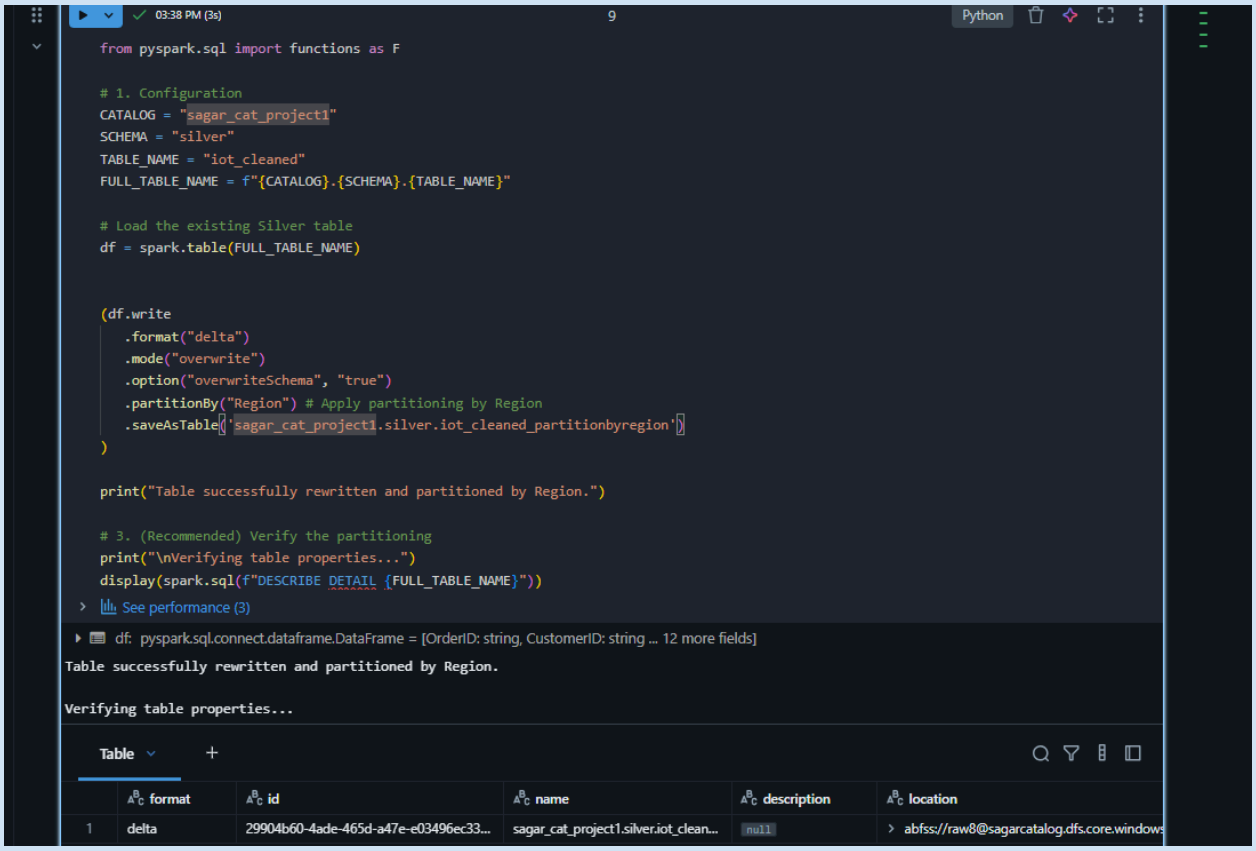
Normalizing storeid and product cartegories->





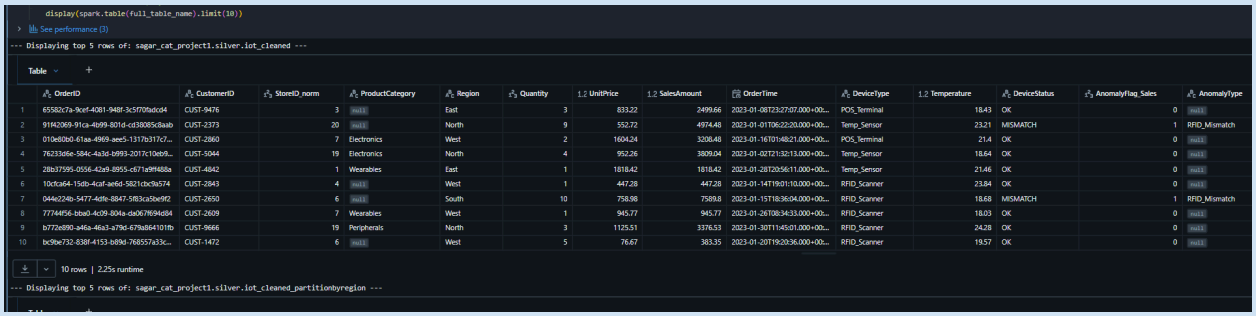
Finally saving in silver schema of my catalog

Partioning by region and zordering->



Have a look on all the tables that is created in silver region

iot_cleaned->



```
display(spark.table(full_table_name).limit(10))
```

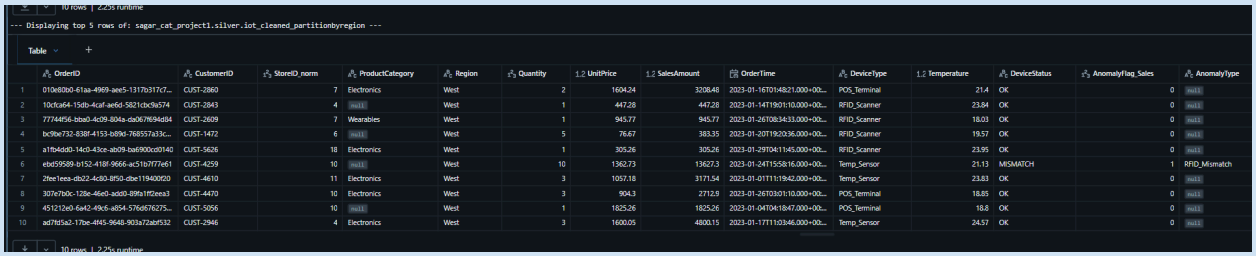
```
--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned ---
```

	OrderID	CustomerID	StoreID_norm	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag_Sales	AnomalyType
1	65582c7a-9cef-4081-948f-3c5f70fdcd4	CUST-9476	3	null	East	3	833.22	2499.66	2023-01-08T23:27:07.000+00:...	POS_Terminal	18.43	OK	0	null
2	91f42069-91ca-4b99-801d-cd38085c8aab	CUST-2373	20	null	North	9	552.72	4974.48	2023-01-01T06:22:20.000+00:...	Temp_Sensor	23.21	MISMATCH	1	RFID_Mismatch
3	010e80b0-61aa-4969-aee5-1317b317c7...	CUST-2860	7	Electronics	West	2	1604.24	3208.48	2023-01-16T01:48:21.000+00:...	POS_Terminal	21.4	OK	0	null
4	76233d6e-584c-4a3d-b993-2017c10eb9...	CUST-5044	19	Electronics	North	4	952.26	3809.04	2023-01-02T21:32:13.000+00:...	Temp_Sensor	18.64	OK	0	null
5	28b37595-0556-42a9-8955-c671a9f468a	CUST-4842	1	Wearables	East	1	1818.42	1818.42	2023-01-28T20:56:11.000+00:...	Temp_Sensor	21.46	OK	0	null
6	10cfca64-15db-4caf-ae6d-5821cbc9a574	CUST-2843	4	null	West	1	447.28	447.28	2023-01-14T19:01:10.000+00:...	RFID_Scanner	23.84	OK	0	null
7	044e224b-5477-4dfe-8847-5f83ca5be9f2	CUST-2650	6	null	South	10	758.98	7589.8	2023-01-15T18:36:04.000+00:...	RFID_Scanner	18.68	MISMATCH	1	RFID_Mismatch
8	77744f56-bba0-4c09-804a-da067f694d84	CUST-2609	7	Wearables	West	1	945.77	945.77	2023-01-26T08:34:33.000+00:...	RFID_Scanner	18.03	OK	0	null
9	b772e890-a46a-46a3-a78d-679a864101fb	CUST-9666	19	Peripherals	North	3	1125.51	3376.53	2023-01-30T11:45:01.000+00:...	RFID_Scanner	24.28	OK	0	null
10	bc9be732-838f-4153-b89d-768557a33c...	CUST-1472	6	null	West	5	76.67	383.35	2023-01-20T19:20:36.000+00:...	RFID_Scanner	19.57	OK	0	null

10 rows | 2.25s runtime

```
--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_partitionbyregion ---
```

after partitioning->

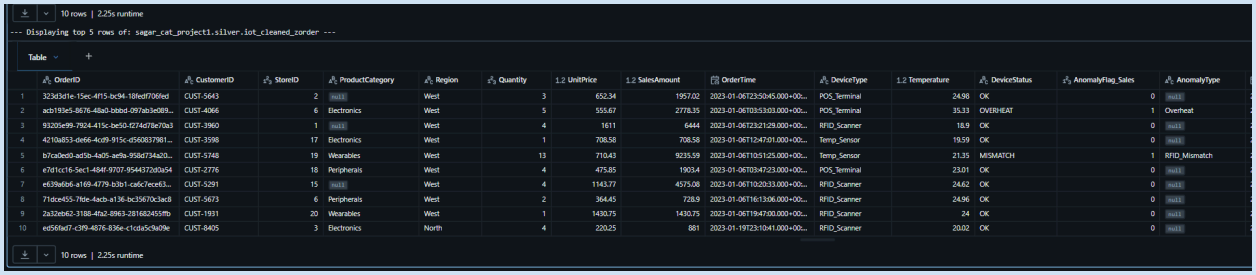


10 rows | 2.25s runtime

```
--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_partitionbyregion ---
```

	OrderID	CustomerID	StoreID_norm	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag_Sales	AnomalyType
1	010e80b0-61aa-4969-aee5-1317b317c7...	CUST-2860	7	Electronics	West	2	1604.24	3208.48	2023-01-16T01:48:21.000+00:...	POS_Terminal	21.4	OK	0	null
2	10cfca64-15db-4caf-ae6d-5821cbc9a574	CUST-2843	4	null	West	1	447.28	447.28	2023-01-14T19:01:10.000+00:...	RFID_Scanner	23.84	OK	0	null
3	77744f56-bba0-4c09-804a-da067f694d84	CUST-2609	7	Wearables	West	1	945.77	945.77	2023-01-26T08:34:33.000+00:...	RFID_Scanner	18.03	OK	0	null
4	bc9be732-838f-4153-b89d-768557a33c...	CUST-1472	6	null	West	5	76.67	383.35	2023-01-20T19:20:36.000+00:...	RFID_Scanner	19.57	OK	0	null
5	a1fb4dd0-14c0-43ce-ab09-ba6900cd0140	CUST-5626	18	Electronics	West	1	305.26	305.26	2023-01-29T04:11:45.000+00:...	RFID_Scanner	23.95	OK	0	null
6	ebd59589-b152-418f-9666-ac51b7f77e61	CUST-4259	10	null	West	10	1362.73	13627.3	2023-01-24T15:58:16.000+00:...	Temp_Sensor	21.13	MISMATCH	1	RFID_Mismatch
7	2fee1eea-db22-4c80-8f50-dbe119400f20	CUST-4610	11	Electronics	West	3	1057.18	3171.54	2023-01-01T11:19:42.000+00:...	Temp_Sensor	23.83	OK	0	null
8	307e7b0c-128e-46e0-add0-89fa1ff2eea3	CUST-4470	10	Electronics	West	3	904.3	2712.9	2023-01-26T03:01:10.000+00:...	POS_Terminal	18.85	OK	0	null
9	451212e0-6a42-49c6-a854-576d676275...	CUST-5056	10	null	West	1	1825.26	1825.26	2023-01-04T04:18:47.000+00:...	POS_Terminal	18.8	OK	0	null
10	ad7fd5a2-17be-4f45-9648-903a72abf532	CUST-2946	4	Electronics	West	3	1600.05	4800.15	2023-01-17T11:03:46.000+00:...	Temp_Sensor	24.57	OK	0	null

partitioning zorder->



10 rows | 2.25s runtime

```
--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_zorder ---
```

	OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag_Sales	AnomalyType
1	323d3d1e-15ec-4f15-bc94-18fedf706fed	CUST-5643	2	null	West	3	652.34	1957.02	2023-01-06T23:50:45.000+00:...	POS_Terminal	24.98	OK	0	null
2	acb193e5-8676-48a0-bbbd-097ab3e089...	CUST-4066	6	Electronics	West	5	555.67	2778.35	2023-01-06T03:53:03.000+00:...	POS_Terminal	35.33	OVERHEAT	1	Overheat
3	93205e99-7924-415c-be50-f274d78e70a3	CUST-3960	1	null	West	4	1611	6444	2023-01-06T23:21:29.000+00:...	RFID_Scanner	18.9	OK	0	null
4	4210a853-de66-4cd9-915c-d560837981...	CUST-3598	17	Electronics	West	1	708.58	708.58	2023-01-06T12:47:01.000+00:...	Temp_Sensor	19.59	OK	0	null
5	b7ca0ed0-ad5b-4a05-ae9a-958d734a20...	CUST-5748	19	Wearables	West	13	710.43	9235.59	2023-01-06T10:51:25.000+00:...	Temp_Sensor	21.35	MISMATCH	1	RFID_Mismatch
6	e7d1cc16-5ec1-484f-9707-9544372d0a54	CUST-2776	18	Peripherals	West	4	475.85	1903.4	2023-01-06T03:47:23.000+00:...	POS_Terminal	23.01	OK	0	null
7	e639a6b6-a169-4779-b3b1-ca6c7ece63...	CUST-5291	15	null	West	4	1143.77	4575.08	2023-01-06T10:20:33.000+00:...	RFID_Scanner	24.62	OK	0	null
8	71dce455-7fde-4acb-a136-bc35670c3ac8	CUST-5673	6	Peripherals	West	2	364.45	728.9	2023-01-06T16:13:06.000+00:...	RFID_Scanner	24.96	OK	0	null
9	2a32eb62-3188-4fa2-8963-28168245f5ff0	CUST-1931	20	Wearables	West	1	1430.75	1430.75	2023-01-06T19:47:00.000+00:...	RFID_Scanner	24	OK	0	null
10	ed56fad7-c3f9-4676-836e-c1cda5c9a09e	CUST-8405	3	Electronics	North	4	220.25	881	2023-01-19T23:10:41.000+00:...	RFID_Scanner	20.02	OK	0	null

10 rows | 2.25s runtime

Aggregation & Enrichment Layer (Gold)->

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window



CATALOG = "sagar_cat_project1"
SILVER_SCHEMA = f"{CATALOG}.silver"
GOLD_SCHEMA = f"{CATALOG}.gold"
BASE_TABLE = f"{SILVER_SCHEMA}.iot_cleaned"

# Create the Gold schema if it doesn't exist
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD_SCHEMA}")
```

```
# Load the base Silver table
base_df = spark.table(BASE_TABLE)



# Standardize column names to ensure consistency
# (Handles cases like 'sales_amount' vs 'SalesAmount' and 'StoreID_norm' vs 'StoreID')
if "sales_amount" in base_df.columns and "SalesAmount" not in base_df.columns:
    base_df = base_df.withColumnRenamed("sales_amount", "SalesAmount")
if "StoreID_norm" in base_df.columns and "StoreID" not in base_df.columns:
    base_df = base_df.withColumnRenamed("StoreID_norm", "StoreID")

# Ensure correct data types for calculations
base_df = (base_df
           .withColumn("OrderTime", F.to_timestamp("OrderTime"))
           .withColumn("SalesAmount", F.col("SalesAmount").cast("double"))
           .withColumn("Quantity", F.col("Quantity").cast("int"))
          )

# Derive common date and month grains for aggregation
base_df = (base_df
           .withColumn("order_date", F.to_date("OrderTime"))
           .withColumn("order_month", F.date_trunc("month", "OrderTime"))
          )

# ----------------------------
# 1) KPI: Daily and Monthly Sales per Region
# ----------------------------
print("Creating sales KPIs per region...")

# Daily aggregation
daily_sales = (base_df
    .groupBy("order_date", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
```

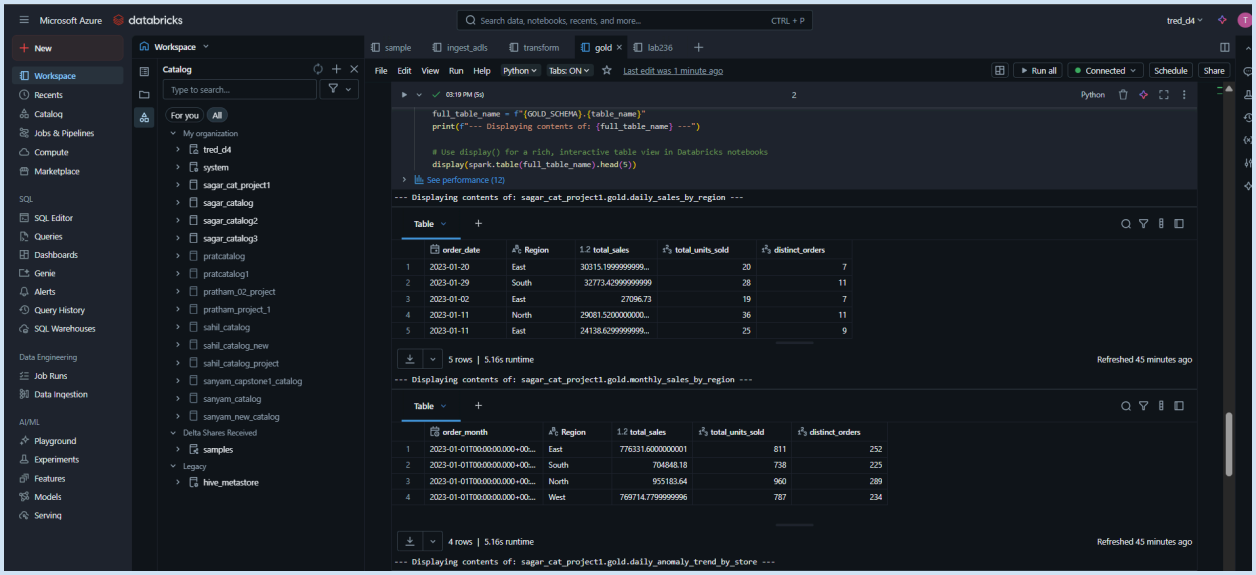
```
    )
)
(daily_sales.write.format("delta").mode("overwrite")
 .saveAsTable(f"{GOLD_SCHEMA}.daily_sales_by_region"))

# Monthly aggregation
monthly_sales = (base_df
    .groupBy("order_month", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)
(monthly_sales.write.format("delta").mode("overwrite")
 .saveAsTable(f"{GOLD_SCHEMA}.monthly_sales_by_region"))

# ----------------------------
# 2) KPI: Device Anomaly Trend per Store
# ----------------------------
print("Creating device anomaly trend KPI...")

anomaly_trend = (base_df
    .groupBy("order_date", "StoreID")
    .agg(
        F.sum("AnomalyFlag_Sales").alias("anomaly_count"),
        F.count("OrderID").alias("total_transactions")
    )
    .withColumn("anomaly_rate", F.col("anomaly_count") / F.col("total_transactions"))
)
(anomaly_trend.write.format("delta").mode("overwrite")
 .saveAsTable(f"{GOLD_SCHEMA}.daily_anomaly_trend_by_store"))

# ----------------------------
# 3) KPI: Conversion Rate Impact from Device Failures
# ----------------------------
print("Creating conversion rate impact KPI...")
```

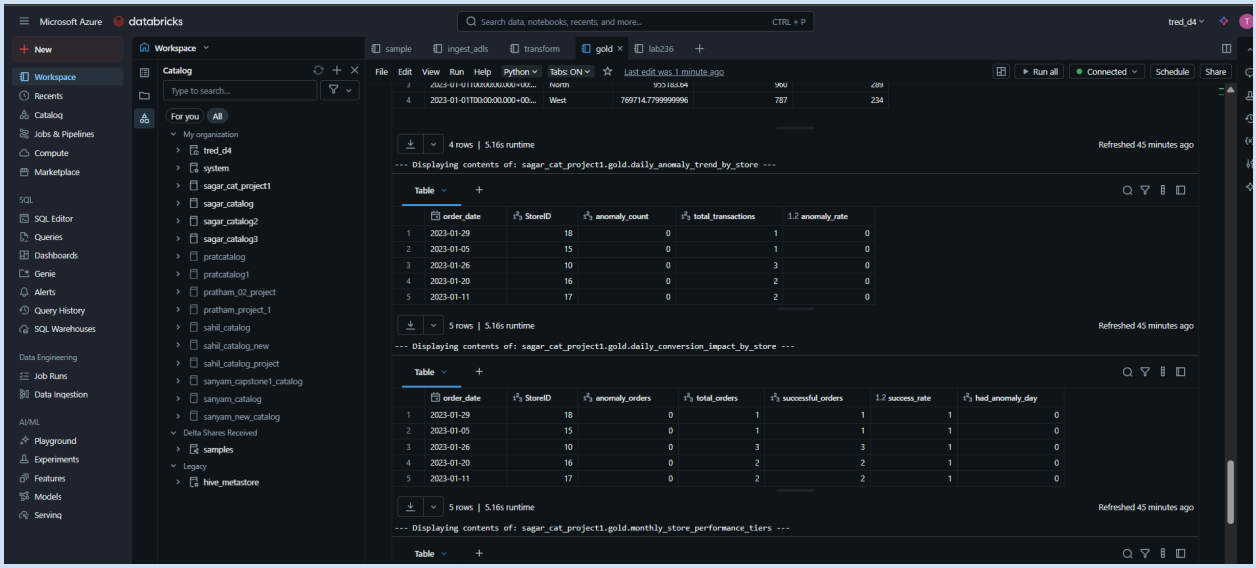
```
# We define "conversion" as the ratio of successful orders to total orders at a store-day level
daily_impact = (base_df
   .groupBy("order_date", "StoreID")
   .agg(
      F.sum(F.when(F.col("AnomalyFlag_Sales") == 1, 1).otherwise(0)).alias("anomaly_orders"),
      F.count("OrderID").alias("total_orders")
   )
   .withColumn("successful_orders", F.col("total_orders") - F.col("anomaly_orders"))
   .withColumn("success_rate", F.col("successful_orders") / F.col("total_orders"))
   .withColumn("had_anomaly_day", F.when(F.col("anomaly_orders") > 0, 1).otherwise(0))
)
(daily_impact.write.format("delta").mode("overwrite")
 .saveAsTable(f"{GOLD_SCHEMA}.daily_conversion_impact_by_store"))

# ----------------------------
# 4) Enrichment: Tag Stores into Performance Tiers
# ----------------------------
print("Enriching stores with performance tiers...")

# Calculate total sales per store for each month
store_monthly_sales = base_df.groupBy("order_month",
"StoreID").agg(F.sum("SalesAmount").alias("total_sales"))

# Use a window function to rank stores within each month
w_spec = Window.partitionBy("order_month").orderBy(F.col("total_sales").desc())
store_ranks = store_monthly_sales.withColumn("rank", F.percent_rank().over(w_spec))

# Assign tiers based on percentile rank
store_tiers = store_ranks.withColumn("performance_tier",
   F.when(F.col("rank") <= 0.2, "High")
    .when((F.col("rank") > 0.2) & (F.col("rank") <= 0.8), "Medium")
    .otherwise("Low")
)
(store_tiers.select("order_month", "StoreID", "total_sales", "performance_tier")
 .write.format("delta").mode("overwrite")
 .saveAsTable(f"{GOLD_SCHEMA}.monthly_store_performance_tiers"))

# ----------------------------
```

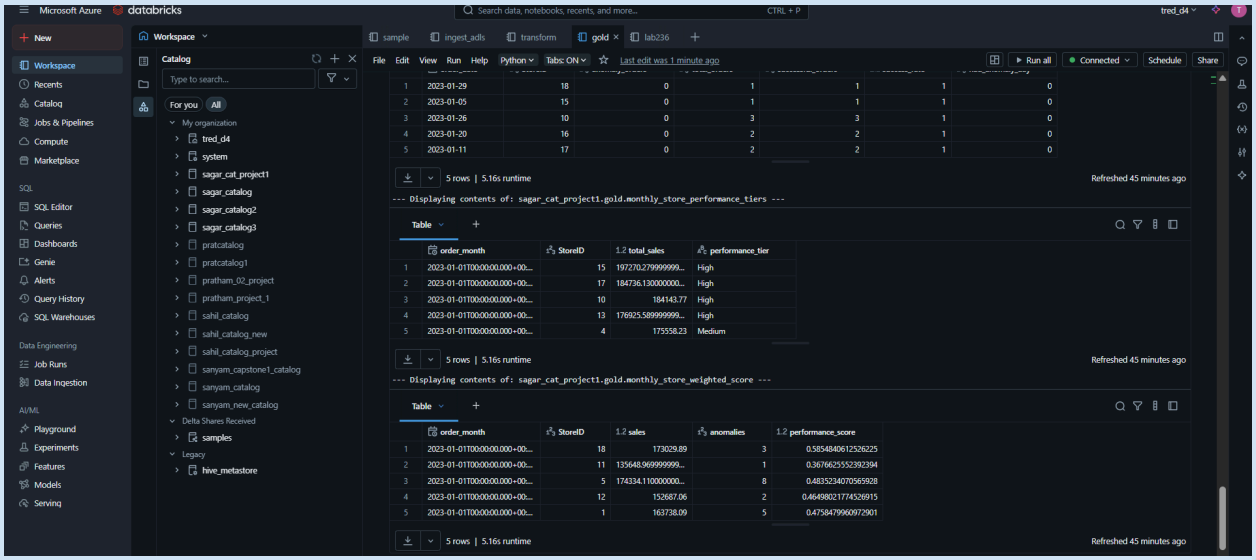
```
# 5) Enrichment: Compute Weighted Sales vs. Anomalies Score
# ----------------------------
print("Enriching stores with a weighted performance score...")

# Get monthly sales and anomaly counts per store
monthly_kpis = base_df.groupBy("order_month", "StoreID").agg(
    F.sum("SalesAmount").alias("sales"),
    F.sum("AnomalyFlag_Sales").alias("anomalies")
)

# Define a window to normalize scores across all stores within a month
w_monthly = Window.partitionBy("order_month")

# Calculate min-max normalized scores for sales and anomalies
normalized_kpis = (monthly_kpis
    .withColumn("max_sales", F.max("sales").over(w_monthly))
    .withColumn("min_sales", F.min("sales").over(w_monthly))
    .withColumn("max_anomalies", F.max("anomalies").over(w_monthly))
    .withColumn("min_anomalies", F.min("anomalies").over(w_monthly))
    .withColumn("sales_norm",
        F.when(F.col("max_sales") == F.col("min_sales"), 1.0)
         .otherwise((F.col("sales") - F.col("min_sales")) / (F.col("max_sales") - F.col("min_sales")))
    )
    .withColumn("anomalies_norm",
        F.when(F.col("max_anomalies") == F.col("min_anomalies"), 0.0)
         .otherwise((F.col("anomalies") - F.col("min_anomalies")) / (F.col("max_anomalies") -
F.col("min_anomalies")))
    )
)

# Compute the final weighted score (e.g., 80% weight for sales, 20% for anomalies)
weighted_score = normalized_kpis.withColumn(
    "performance_score",
    (0.8 * F.col("sales_norm")) - (0.2 * F.col("anomalies_norm"))
)

(weighted_score.select("order_month", "StoreID", "sales", "anomalies", "performance_score")
 .write.format("delta").mode("overwrite")
```

```
.saveAsTable(f"{GOLD_SCHEMA}.monthly_store_weighted_score"))

print("\n--- Gold Layer Generation Complete ---")
```

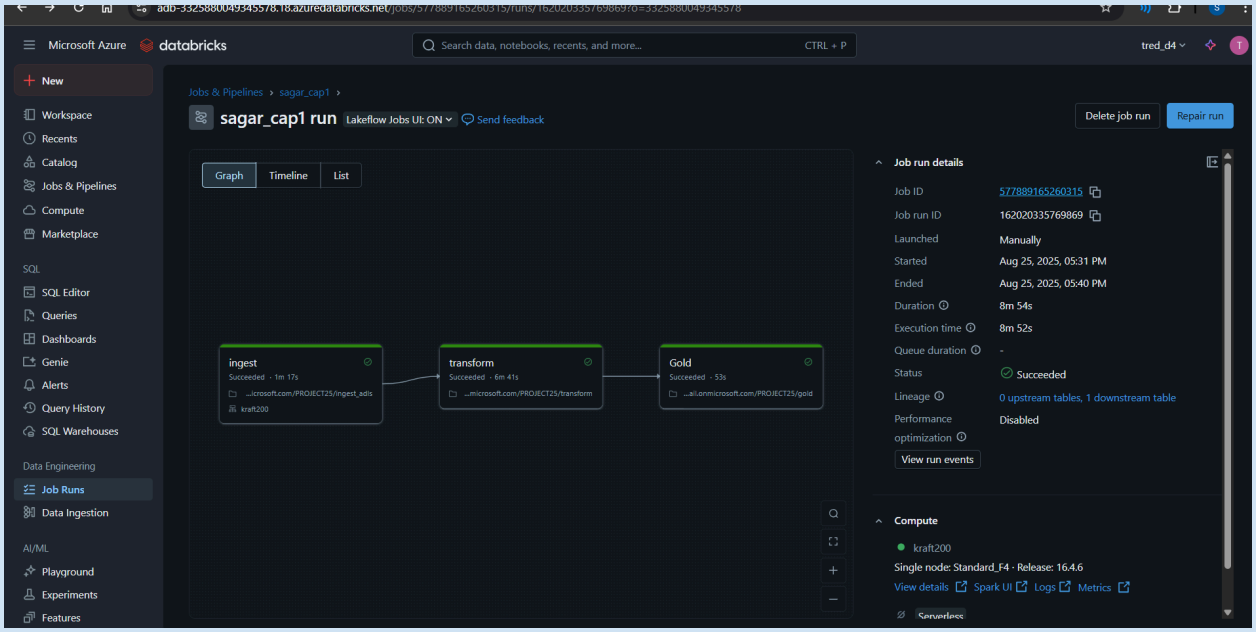
Finally have a look of all the tables that is created.



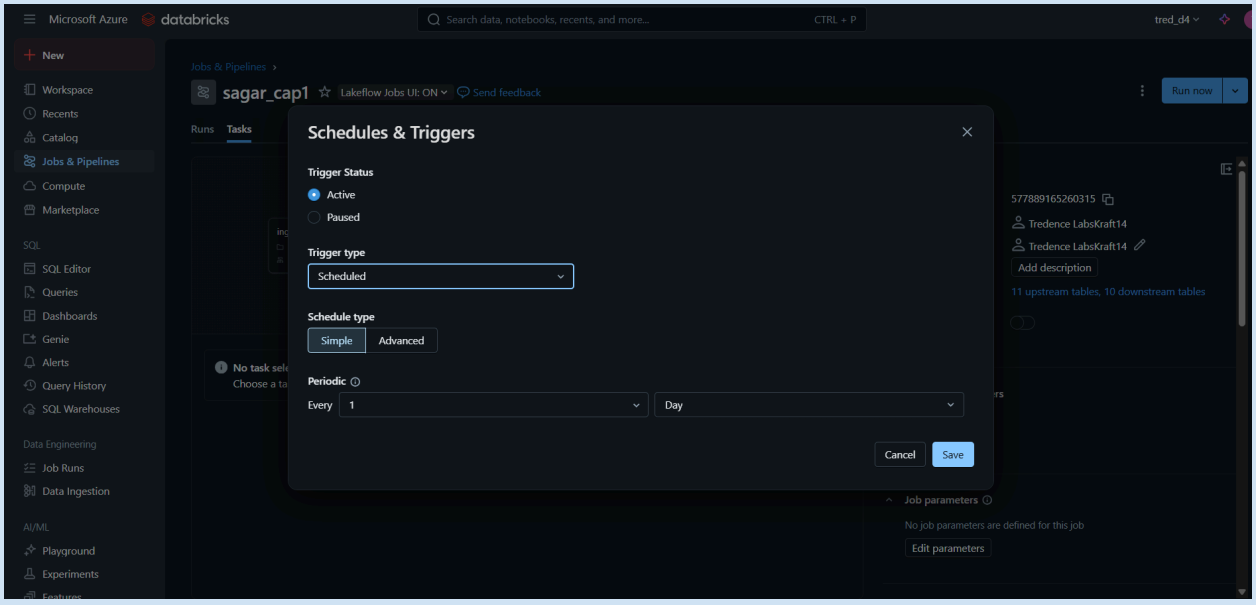

Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
tred_d4
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Models
Serving
Catalog
Type to search...
For you
All
My organization
tred_d4
system
sagar_cat_project1
sagar_catalog
sagar_catalog2
sagar_catalog3
pratcatalog
pratcatalog1
pratham_02_project
pratham_project_1
sahil_catalog
sahil_catalog_new
sahil_catalog_project
sanyam_capstone1_catalog
sanyam_catalog
sanyam_new_catalog
Delta Shares Received
samples
Legacy
hive_metastore
sample
ingest_adls
transform
gold
lab236
File Edit View Run Help
Python
Tabs: ON
Last edit was 1 minute ago
Run all
Connected
Schedule
Share
4 rows | 5.16s runtime
Refreshed 45 minutes ago
--- Displaying contents of: sagar_cat_project1.gold.daily_anomaly_trend_by_store ---
Table
order_date
StoreID
anomaly_count
total_transactions
anomaly_rate
2023-01-29 18 0 1 0
2023-01-05 15 0 1 0
2023-01-26 10 0 3 0
2023-01-20 16 0 2 0
2023-01-11 17 0 2 0
5 rows | 5.16s runtime
Refreshed 45 minutes ago
--- Displaying contents of: sagar_cat_project1.gold.daily_conversion_impact_by_store ---
Table
order_date
StoreID
anomaly_orders
total_orders
successful_orders
success_rate
had_anomaly_day
2023-01-29 18 0 1 1 1 0
2023-01-05 15 0 1 1 1 0
2023-01-26 10 0 3 3 1 0
2023-01-20 16 0 2 2 1 0
2023-01-11 17 0 2 2 1 0
5 rows | 5.16s runtime
Refreshed 45 minutes ago
--- Displaying contents of: sagar_cat_project1.gold.monthly_store_performance_tiers ---
Table


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
tred_d4
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Models
Serving
Catalog
Type to search...
For you
All
My organization
tred_d4
system
sagar_cat_project1
sagar_catalog
sagar_catalog2
sagar_catalog3
pratcatalog
pratcatalog1
pratham_02_project
pratham_project_1
sahil_catalog
sahil_catalog_new
sahil_catalog_project
sanyam_capstone1_catalog
sanyam_catalog
sanyam_new_catalog
Delta Shares Received
samples
Legacy
hive_metastore
sample
ingest_adls
transform
gold
lab236
File Edit View Run Help
Python
Tabs: ON
Last edit was 1 minute ago
Run all
Connected
Schedule
Share
2023-01-29 18 0 1 1 1 0
2023-01-05 15 0 1 1 1 0
2023-01-26 10 0 3 3 1 0
2023-01-20 16 0 2 2 1 0
2023-01-11 17 0 2 2 1 0
5 rows | 5.16s runtime
Refreshed 45 minutes ago
--- Displaying contents of: sagar_cat_project1.gold.monthly_store_performance_tiers ---
Table
order_month
StoreID
total_sales
performance_tier
2023-01-01T00:00:00.000+00:... 15 197270.279999999... High
2023-01-01T00:00:00.000+00:... 17 184736.130000000... High
2023-01-01T00:00:00.000+00:... 10 184143.77 High
2023-01-01T00:00:00.000+00:... 13 176925.589999999... High
2023-01-01T00:00:00.000+00:... 4 175558.23 Medium
5 rows | 5.16s runtime
Refreshed 45 minutes ago
--- Displaying contents of: sagar_cat_project1.gold.monthly_store_weighted_score ---
Table
order_month
StoreID
sales
anomalies
performance_score
2023-01-01T00:00:00.000+00:... 18 173029.89 3 0.585484061526225
2023-01-01T00:00:00.000+00:... 11 135648.969999999... 1 0.367662555292394
2023-01-01T00:00:00.000+00:... 5 174334.110000000... 8 0.483523407056928
2023-01-01T00:00:00.000+00:... 12 152687.06 2 0.464980217745269 15
2023-01-01T00:00:00.000+00:... 1 163738.09 5 0.475847996097290 1
5 rows | 5.16s runtime
Refreshed 45 minutes ago

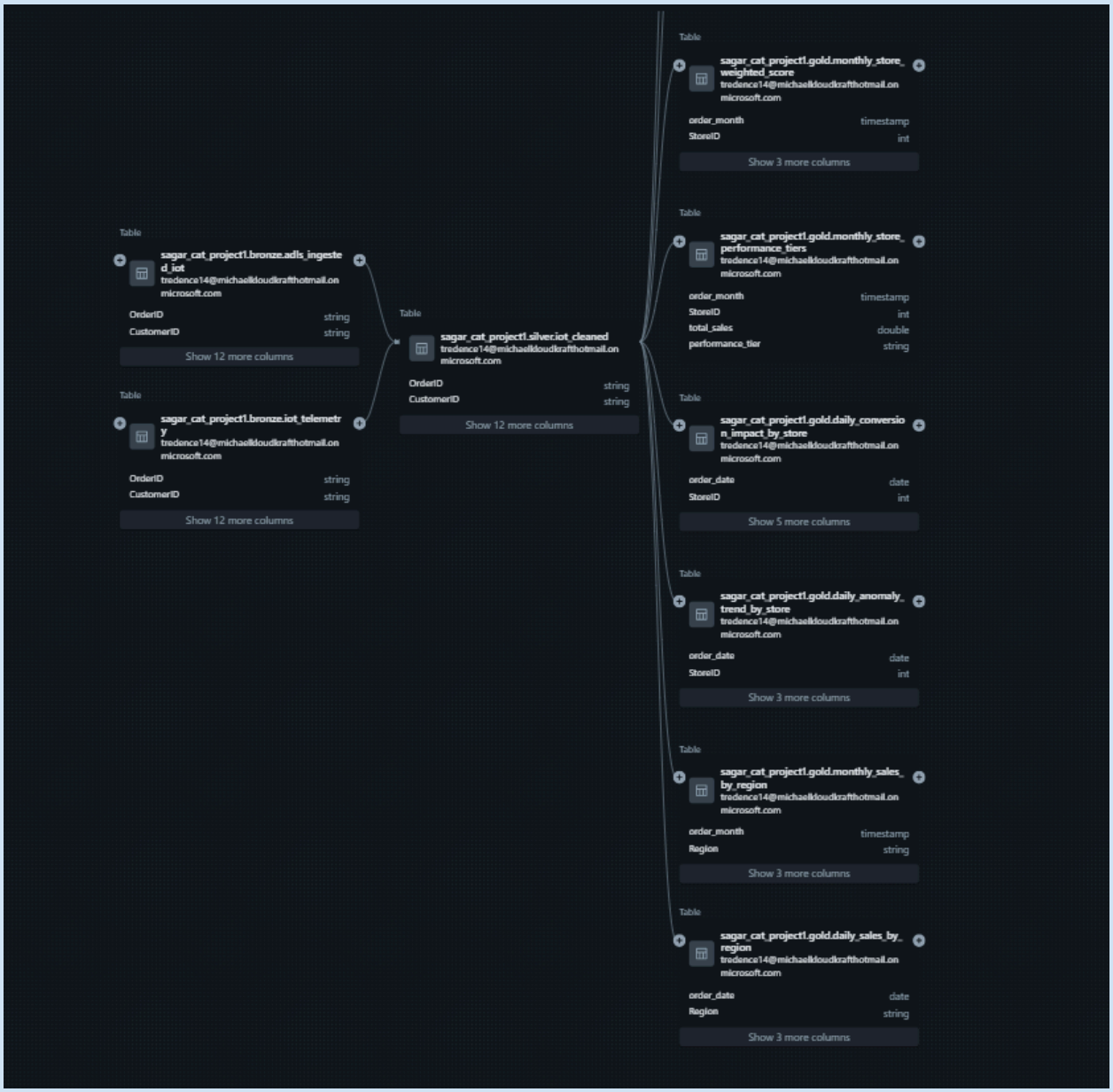
JOBS creation->



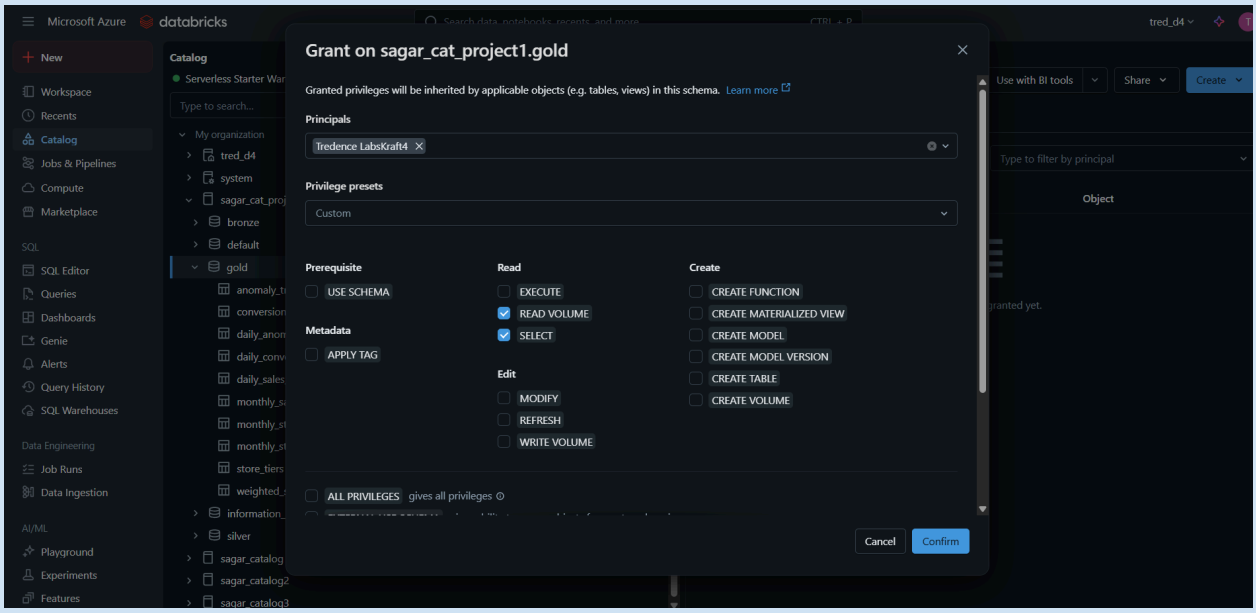
scheduling for regular ingestion and updation of tables->



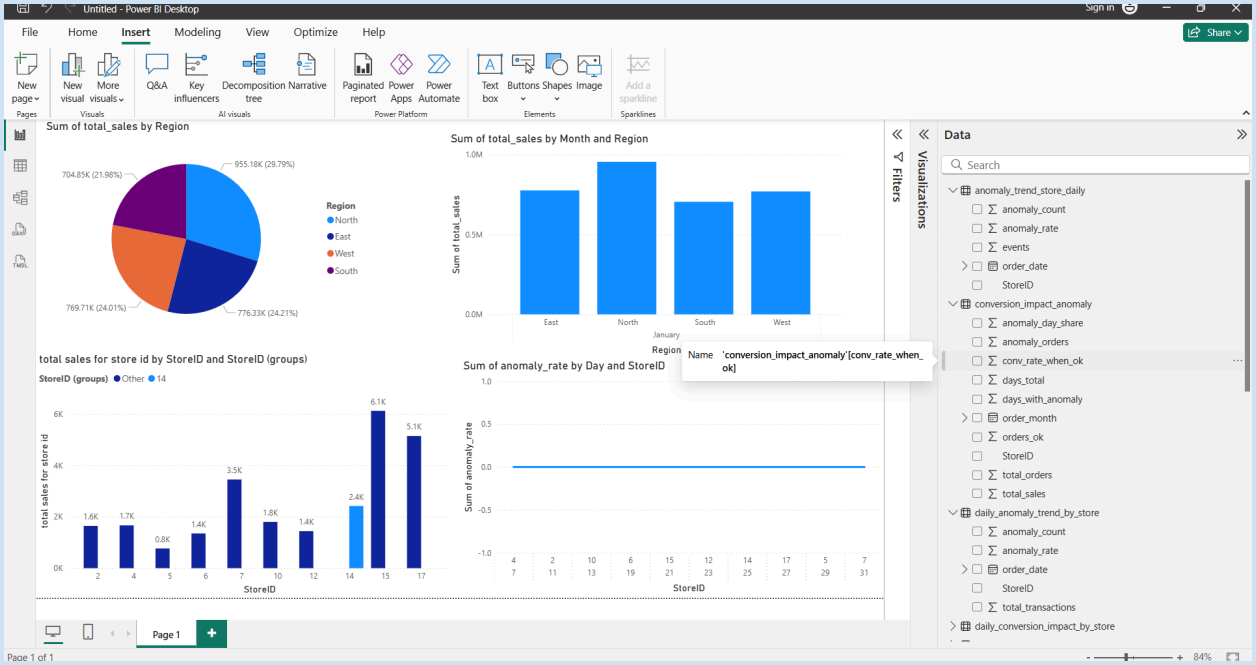
Lineage Graph->

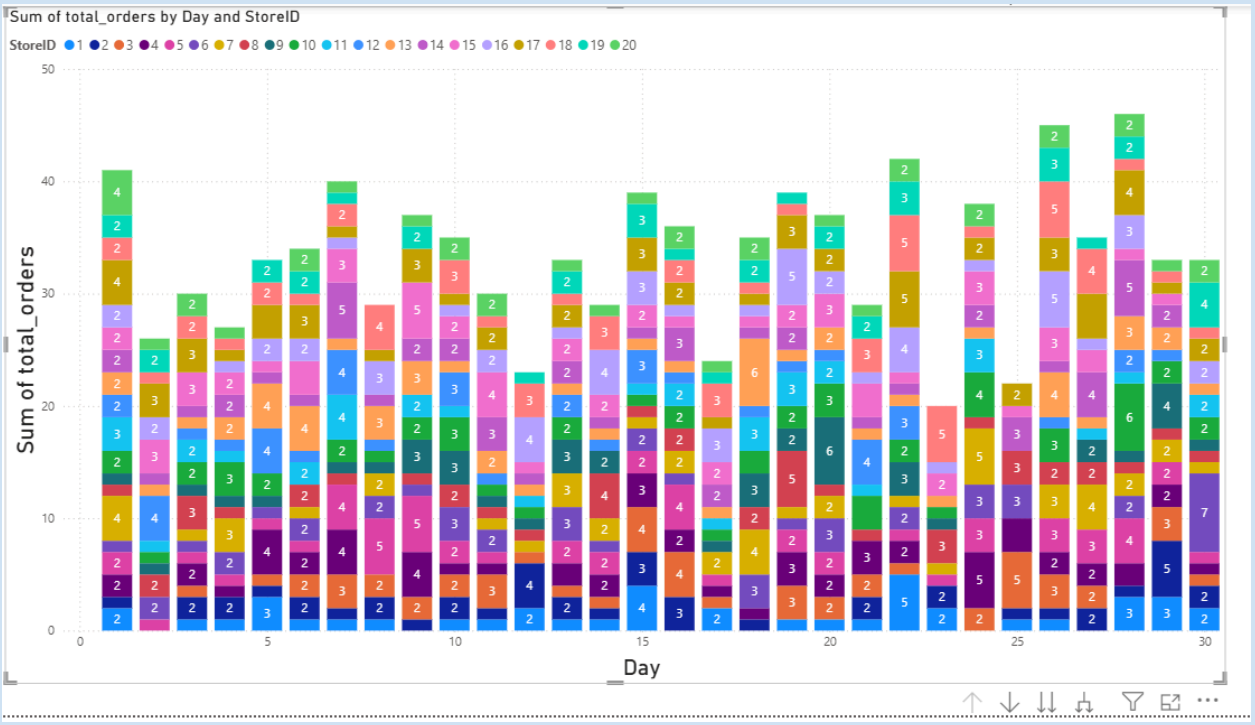


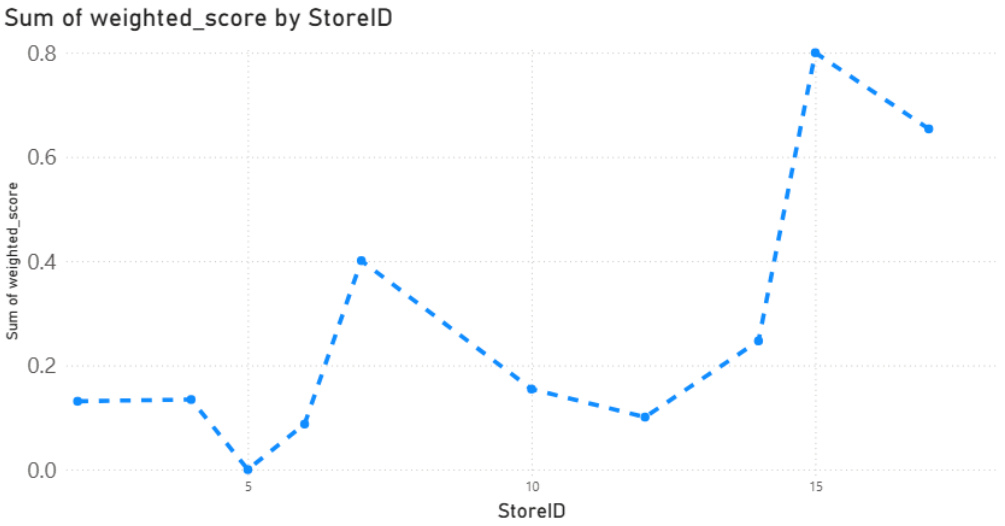
Giving bi user acces to my gold tables->

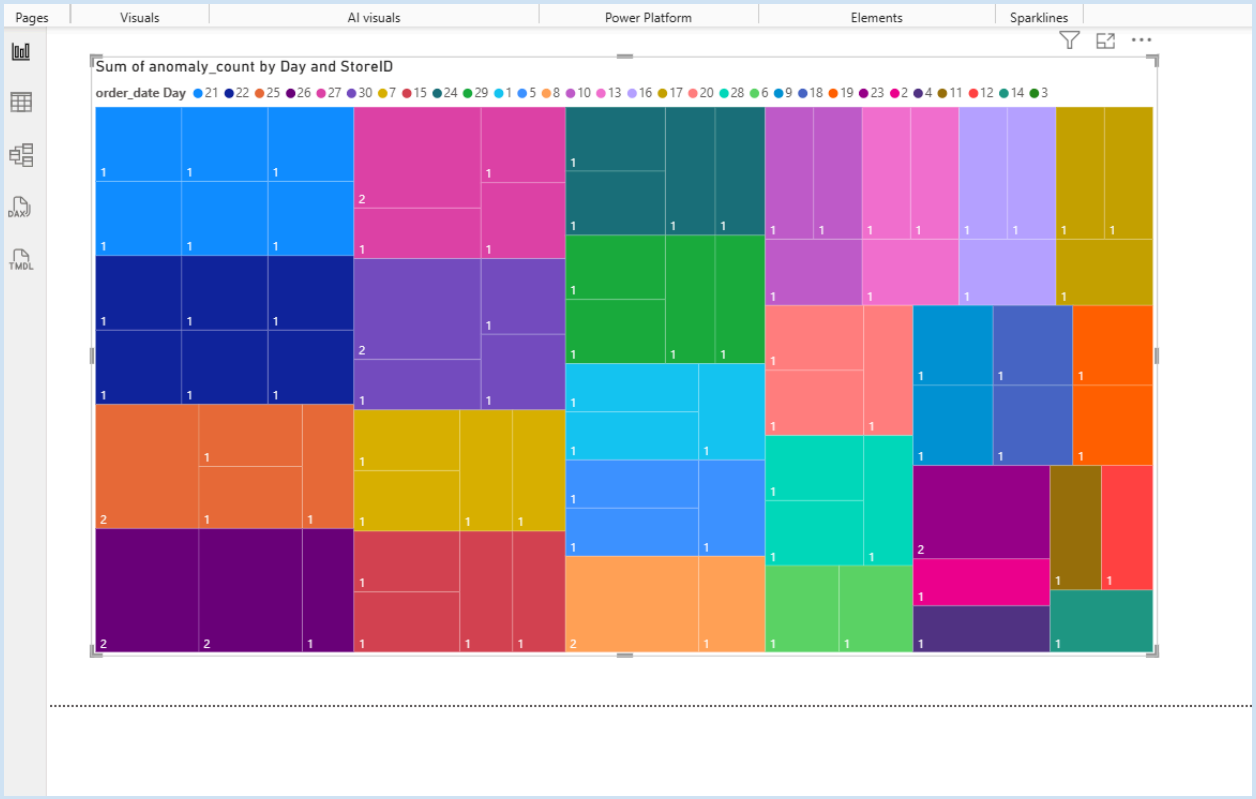


Connecting through PowerBi->




Sum of total_orders by Day and StoreID
StoreID 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Sum of total_orders
Day
0
10
20
30
40
50
0
5
10
15
20
25
30


Sum of weighted_score by StoreID
Sum of weighted_score
StoreID
0.0
0.2
0.4
0.6
0.8
5
10
15


Pages
Visuals
AI visuals
Power Platform
Elements
Sparklines
Sum of anomaly_count by Day and StoreID
order_date Day 21 22 25 26 27 30 7 15 24 29 1 5 8 10 13 16 17 20 28 6 9 18 19 23 2 4 11 12 14 3